

MONOTONICITY AS AN ARCHITECTURAL BIAS FOR ROBUST LANGUAGE MODELS

Anonymous authors

Paper under double-blind review

ABSTRACT

Large language models (LLMs) remain strikingly brittle under adversarial prompts and jailbreak attacks: small, carefully chosen perturbations can produce large and unpredictable changes in internal representations and generated outputs. We introduce *monotonicity* as an architectural inductive bias for improving the robustness of Transformer language models. Rather than constraining the entire network, we impose monotonicity selectively within feed-forward sublayers, where semantic representations are nonlinearly refined, while leaving attention unconstrained to preserve contextual reasoning, negation, and contradiction. This separation yields *monotone language models* whose semantic refinement is order-preserving without globally restricting Transformer expressivity. On an encoder-decoder model (T5), this simple structural constraint substantially improves adversarial robustness while largely preserving task performance: gradient-based attack success rates fall from 60% for an identically fine-tuned unconstrained baseline (69% for the pretrained model) to 17%, with a modest reduction in summarization quality (a 2–5% relative ROUGE-L decrease on two benchmarks, consistent across five training seeds on full test sets), without adversarial training, additional data, or modified training objectives. We further show that the effect transfers to the decoder-only Pythia-1.4B architecture, but only when the constraint is placed along the relevant propagation paths. Constraining the feed-forward input projection alone is ineffective, whereas constraining both projections reduces mean attack success from 69.0% to 42.5% across three seeds, at a markedly larger cost in clean perplexity; the advantage persists under random-substitution and gradient-free query attacks, arguing against gradient masking as the explanation. Together, these results identify monotonicity as a practical structural prior for robust language modeling and show that its effectiveness depends critically on where constraints are imposed within the architecture.

1 INTRODUCTION

Large language models (LLMs) achieve remarkable performance across language tasks, yet their behavior remains surprisingly brittle: carefully crafted prompts, jailbreaks, and even small targeted perturbations can push otherwise well-aligned models toward unsafe or unintended outputs. This vulnerability is not merely a failure of alignment; it reflects a deeper property of neural language models, whose representations evolve in extremely high-dimensional spaces where small input changes can be amplified through successive layers, a sensitivity linked in part to the large Lipschitz constants of Transformers (Newhouse et al., 2025). Existing approaches to robustness (Wei et al., 2023; Su et al., 2024) largely rely on training-time interventions such as alignment objectives (Cao et al., 2024) and adversarial training (Howe et al., 2025), or on inference-time defenses such as output filtering (Khachaturov & Mullins, 2025); while effective in many settings, they are typically reactive, suppressing undesirable behavior after the underlying vulnerability has emerged. This raises a fundamental question: *can robustness be built into the architecture itself, rather than added only after vulnerabilities are discovered?*

We explore *monotonicity* as such a structural inductive bias. Monotonicity imposes directional consistency: strengthening an input along an ordered dimension cannot cause the corresponding downstream representation to move in the opposite direction. This property has a long history in machine learning (Gupta et al., 2016; Runje & Shankaranarayana, 2023) and in control theory, where

monotone systems admit tractable reasoning about stability and perturbation propagation in high-dimensional settings (Smith, 1995; Michel et al., 2015; Ito et al., 2014; Rantzer & Bernhardsson, 2014). It has a natural interpretation in language models: adding supporting evidence should not weaken a conclusion, adding safety constraints should not produce a less constrained internal state, and increasing the stated severity of a situation should not make the model more permissive. A sequence of prompts that progressively strengthens a claim, “*The medication has side effects*” $\preceq$ “*The medication has severe side effects*” $\preceq$ “*The medication has severe side effects and requires medical supervision*”, induces a natural partial order over meaning, and ideally the model’s internal representations should preserve this ordering rather than allow stronger information to produce weaker or contradictory downstream states. Contemporary language models often violate this expectation, and we argue that the instability lies in how semantic information is transformed inside the network, not only in the input space.

This perspective suggests a structural source of brittleness. Transformers alternate between contextual aggregation through attention, which is naturally suited to explicit contextual operations such as negation, contrast, and exception, and nonlinear feed-forward transformations, which are largely unconstrained and can amplify, suppress, or reverse semantic features in ways that are difficult to predict; we hypothesize that these unconstrained reversals create exploitable directions through which small adversarial perturbations can be amplified. Our motivation mirrors *monotone Boolean circuits* (Jukna, 2012), in which negation is excluded from the internal computation and must be introduced explicitly at the input: by enforcing monotonicity only during semantic refinement, we encourage negation, contradiction, and constraint relaxation to arise through attention rather than through fragile cancellation inside nonlinear transformations. The resulting Transformer is not globally monotone, nor does the constraint eliminate all adversarial behavior; it imposes structure precisely where semantic features are refined.

Our central finding is that monotonicity can be incorporated into Transformer architectures at a modest cost in task performance in the encoder–decoder setting, and at a markedly larger cost in clean perplexity in the decoder-only setting, while the resulting models achieve substantially greater robustness in both settings, without adversarial training, additional data, modified training objectives, or heuristic defenses (whether the robustness gain stems from the constraint or from its initialization remains open; Appendix A.10 settles this only for clean quality). Selectively imposed monotonicity is thus a practical inductive bias at an architecture-dependent rather than prohibitive cost, contrary to the assumption that strong structural restrictions must sacrifice optimization or expressive power (see Sartor et al., 2025).

Contributions.

1. **Selective monotonicity for Transformers.** Monotone feed-forward sublayers that enforce order-preserving semantic refinement while leaving attention unconstrained.
2. **Practical monotone language models.** Pretrained Transformers can be converted into monotone counterparts that retain competitive downstream performance in the encoder–decoder setting.
3. **Substantial gains in adversarial robustness.** Monotonicity significantly reduces vulnerability to gradient-based token-level attacks (HotFlip; more weakly, universal triggers); on the decoder-only track the reduction persists under random-substitution and gradient-free query attacks, arguing against gradient masking.
4. **Architecture-dependent constraint placement.** On Pythia-1.4B the benefit transfers under the same constraint scope (both feed-forward projections), constraining the input projection alone is ineffective, and the clean-performance cost is markedly larger.

2 RELATED WORK

Vulnerabilities and defenses. LLMs are highly susceptible to manipulations that bypass alignment, including transferable adversarial suffixes (Zou et al., 2023), position-sensitive (Wang et al., 2025), interpretable (Zhu et al., 2023), many-shot (Anil et al., 2024), and multilingual (Deng et al., 2024) jailbreaks, and prompt injection (Li & Fung, 2025); they also perform poorly under systematic adversarial evaluation (Wang et al., 2021). The token-level attacks we use descend from gradient-guided substitutions (Ebrahimi et al., 2018), universal triggers (Wallace et al., 2019) and their descendants (Shin et al., 2020; Guo et al., 2021), gradient-free query-based substitutions (Jin et al., 2020), and attacks on sequence-to-sequence models (Cheng et al., 2020). Defenses include

smoothing with provable guarantees (Robey et al., 2023), prompt classifiers (Sayeedi et al., 2025), adversarial training (Howe et al., 2025), perplexity filtering (Jain et al., 2023), representation-level interventions (Zou et al., 2024), and certified training (Jia et al., 2019); most remain reactive and training-dependent. Since apparent robustness to gradient-based attacks can be an artifact of obfuscated gradients (Athalye et al., 2018; Carlini et al., 2019; Tramèr et al., 2020), our controls include gradient-free attacks (Section 6.4).

Monotonic networks and structural constraints. Nonnegative-weight sigmoidal networks (Daniëls & Velikova, 2010) and min-max networks of constrained linear units (Sill, 1998) are universal approximators of monotone functions; practical monotone architectures include lattice models (Gupta et al., 2016; You et al., 2017), certified or counterexample-guided constructions (Liu et al., 2020; Sivaraman et al., 2020), integrated positive-derivative networks (Wehenkel & Louppe, 2019), smooth min-max networks (Igel, 2024), and Lipschitz-bounded residual architectures that certify robustness (Nolte et al., 2023; Kitouni et al., 2023); the depth and width price of nonnegative parameters is known (Mikulincer & Reichman, 2022). Monotone classifiers were shown to resist a class of evasion attacks before the LLM era (Íncar Romeo et al., 2018); monotone dynamical systems have order-preserving flows with strong convergence properties (Hirsch & Smith, 2006; Angeli & Sontag, 2003); structured layers (Amos & Kolter, 2017) and Lipschitz control of attention (Kim et al., 2021; Newhouse et al., 2025) are related routes to robustness (see Appendix A.1).

3 MONOTONE TRANSFORMERS

We study monotonicity constraints in a Transformer sequence-to-sequence model $\mathcal{M}_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ from input token sequences to output token sequences, composed of attention layers, feed-forward network (FFN) sublayers, residual connections, and normalization. We define the order along which monotonicity is required, then the constrained sublayer, then how the constraint is enforced; proofs and details are in Appendices A.3 and A.2.

Semantic order. Fix $A \in \mathbb{R}^{p \times d}$ whose rows define interpretable semantic axes. For a hidden state $h \in \mathbb{R}^d$, define semantic coordinates $s := Ah \in \mathbb{R}^p$ and equip $\mathbb{R}^p$ with the standard product order. We define a preorder on $\mathbb{R}^d$ by $h \preceq h' \iff Ah \leq Ah'$ (elementwise) and interpret $h' \succeq h$ as semantically stronger along the chosen axes (when $A = I$, $\preceq$ is the coordinatewise order). In practice, A may be chosen by training p linear probes on ordered prompt pairs or from fixed directions of auxiliary classifiers, and is fixed before imposing constraints. *All models trained in this paper use $A = I$* , so that g below is the model’s own FFN and every FFN weight is constrained; probe-derived directions serve only as a post-hoc diagnostic (Section 6.3). The general construction makes the role of A explicit: with $p < d$ it selects a semantic subspace along which monotonicity holds, leaving $\ker(A)$ unconstrained, and A -monotonicity of the sublayer follows from that of the constrained multilayer perceptron (MLP) g alone (Proposition 3.3).

Definition 3.1 (Monotonicity). A function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is *monotone* (or *coordinatewise monotone*) if $x \leq x'$ implies $f(x) \leq f(x')$ for all $x, x' \in \mathbb{R}^n$; equivalently, f is non-decreasing in each input coordinate with the others held fixed. A function $F : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is *A-monotone* if $h \preceq h' \Rightarrow F(h) \preceq F(h')$, i.e., $Ah \leq Ah' \Rightarrow AF(h) \leq AF(h')$.

Lemma 3.2 (Closure under Composition). *If $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $g : \mathbb{R}^m \rightarrow \mathbb{R}^k$ are monotone, then $g \circ f$ is monotone.*

Semantic-monotone FFN sublayer. Assume A has full row rank and let $A^\dagger \in \mathbb{R}^{d \times p}$ be a fixed right inverse of A (e.g., $A^\dagger = A^\top (AA^\top)^{-1}$), so that $AA^\dagger = I_p$, and define the FFN update

$$F(h) := h + A^\dagger g(Ah), \quad g : \mathbb{R}^p \rightarrow \mathbb{R}^p \text{ coordinatewise monotone.}$$

Proposition 3.3 (Sufficient condition for A -monotonicity). *If g is monotone w.r.t. the product order on $\mathbb{R}^p$, then F is A -monotone. In particular, it suffices that g is an MLP with elementwise non-decreasing activations and elementwise nonnegative weight matrices.*

Proposition 3.4 (Sufficient Condition for FFN Monotonicity). *Let $N : \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_{k+1}}$ be a feed-forward network $y_0 = u$, $y_{i+1} = \sigma(W_i y_i + b_i)$ ($i = 0, \dots, k-1$), $N(u) = W_k y_k + b_k$, with $W_i \in \mathbb{R}^{n_{i+1} \times n_i}$ and σ applied elementwise. If σ is non-decreasing and $W_i \geq 0$ for $i = 0, \dots, k$, then N is monotone.*

Proof. Each affine map $y \mapsto W_i y + b_i$ with $W_i \geq 0$ is monotone, as is the elementwise map $y \mapsto \sigma(y)$ because σ is non-decreasing; the claim follows by Lemma 3.2. $\square$

Such *monotone neural networks* are universal approximators of continuous monotone functions on compact domains when the activation is bounded and sigmoidal (Daniëls & Velikova, 2010); this does not extend to ReLU, for which every output coordinate of a nonnegative-weight network is convex in its input (Runje & Shankaranarayana, 2023), so the ReLU-based monotone FFNs of our T5 experiments are a strictly restricted family; whether this restriction, rather than the initialization, costs clean quality is examined in Appendix A.10.

Definition 3.5 (Monotone Transformer). A Transformer $\mathcal{M}_\theta$ is *A-monotone with respect to its feed-forward sublayers* if each FFN sublayer implements an update $F(h) := h + A^\dagger g(Ah)$ with $A^\dagger$ a fixed right inverse of A and $g : \mathbb{R}^p \rightarrow \mathbb{R}^p$ coordinatewise monotone (e.g., an MLP with elementwise non-decreasing activations and elementwise nonnegative weight matrices). All other components (attention, residuals, normalization) are left unconstrained.

This definition does not assert global monotonicity of the full Transformer; it enforces monotonic refinement at the level of FFN sublayers, where we hypothesize that implicit feature cancellation is most likely to occur (cf. Runje & Shankaranarayana, 2023; Sartor et al., 2025; Nguyen et al., 2023). In the pre-normalized T5 and GPT-NeoX blocks we instantiate, the branch receives $u = \text{LN}(h)$ (layer normalization) and the residual is added outside it, $F(h) = h + g(\text{LN}(h))$, so F itself need not be monotone in h ; the property we enforce, and analyze in Section 5, is the monotonicity of the branch g as a map of its own input u (Remark A.1).

Enforcing the constraint. We constrain all linear maps inside g to be elementwise nonnegative and keep the model’s own activation: the non-decreasing ReLU in the T5 track, and in the Pythia track GELU, which is non-decreasing only outside a small region (Section 4). Each constrained weight matrix is reparameterized as $W = \phi(V)$ with $\phi(v) = \log(1 + \exp(v))$ applied elementwise to an unconstrained V , which maintains $W \geq 0$ throughout optimization without projection. To retain the pretrained weights we initialize $V_{\text{init}} = \phi^{-1}(|W_{\text{pre}}| + \epsilon)$, $\epsilon = 10^{-4}$, which preserves the magnitude of every pretrained weight while flipping the sign of the negative entries; recovering from this sign disruption is the task of the subsequent fine-tuning, and its cost is isolated in Appendix A.10. Pretrained biases are retained unconstrained.

4 EXPERIMENTAL SETUP

Full protocols (hyperparameters, datasets, decoding, attack budgets, controls, and the order-preservation corpus) are in Appendices A.5, A.7, and A.8.

Encoder-decoder track (T5). We instantiate Definition 3.5 with $A = I_d$ on T5-small (Raffel et al., 2020) (6 encoder and 6 decoder layers, $d = 512$, 8 heads, 2048-dimensional FFNs): each FFN sublayer keeps its pretrained architecture, $g(u) = W_o \text{ReLU}(W_i u)$ with $W_i \in \mathbb{R}^{2048 \times 512}$ and $W_o \in \mathbb{R}^{512 \times 2048}$ both constrained through the softplus reparameterization in all 12 blocks (25M of 60.5M parameters, 42%), so that g is a monotone network in the sense of Proposition 3.4. The *baseline* is the same checkpoint fine-tuned identically without the constraint; the *standard* model is the pretrained checkpoint. Fine-tuning uses other summarization corpora (Appendix A.5); all three models are evaluated out of distribution on the full CNN/DailyMail and XSUM test sets over five training seeds.

Attacks (T5). *Universal adversarial triggers* (UATs) (Wallace et al., 2019) are short, input-agnostic token sequences that, when prepended to an input, maximize the model’s loss; we optimize five-token triggers by coordinate-wise search (five restarts, 100 iterations) on 1,000 held-out validation examples and evaluate them on 1,500 disjoint test examples. Because the trigger is prepended and the encoder input is then left-truncated to 512 tokens, the trigger is discarded for articles longer than about 505 tokens, so the T5 trigger results are a lower bound (Appendix A.8). *HotFlip* (Ebrahimi et al., 2018) replaces up to five tokens per example, chosen from the gradient of the loss with respect to token embeddings, with the vocabulary items that maximize the first-order loss increase. Robustness is quantified by the relative increase in cross-entropy loss under attack (*degradation*, averaged over examples), the attack success rate (degradation above 10%), and independent-samples t -tests on per-example degradations; since relative degradation is normalized by each model’s own

clean loss, we also report the absolute increase in nats. For UATs we report the relative negative log-likelihood (NLL) increase.

Order-preservation diagnostic. To test whether sublayer monotonicity yields order preservation along the semantic directions that motivate it, we build a templated corpus of ordered prompt pairs (13 semantic axes on 18 topics; 702 adjacent-level pairs from four-level chains, split by chain into 507 probe-fitting and 195 evaluation pairs), fit $p = 64$ bootstrap logistic-regression probe directions A per model on final-layer representations (mean- or last-token-pooled) of the fitting split, and report at every layer the fraction of probe coordinates with $Ah_{\text{weak}} \leq Ah_{\text{strong}}$ on evaluation pairs; the probe is fit post hoc and plays no role in training.

Decoder-only track (Pythia-1.4B). To test generalization to decoder-only architectures, we repeat the study on Pythia-1.4B (Biderman et al., 2023), a 24-layer GPT-NeoX model with $d = 2048$, fine-tuned and evaluated on the (uncopyrighted) Pile (Gao et al., 2020) with clean performance measured by held-out perplexity. Each block has an attention output projection and an MLP with projections $W_{\text{in}} \in \mathbb{R}^{4d \times d}$ and $W_{\text{out}} \in \mathbb{R}^{d \times 4d}$; since decoder-only blocks lack T5’s encoder–decoder bottleneck, we compare three *constraint scopes* applied in all 24 layers with $A = I$: **MLP-in** (W_{in} only), **MLP-both** (W_{in} and W_{out} , the direct analogue of the T5 scheme), and **MLP-in + Attn-out** (W_{in} and the attention output projection). Two caveats apply: GELU is not globally non-decreasing ($\text{GELU}'(z) < 0$ for $z < -0.752$, with minimum ≈ -0.13), so the constrained Pythia FFNs are monotone only up to this small region and the track tests the same weight constraint rather than an exactly monotone model; and only MLP-both constrains every FFN matrix, so the other scopes are ablations.

After imposing constraints, each model and an unconstrained *baseline* undergo *recovery training* on the Pile, the monotone runs with a larger budget (learning rate 5×10^{-5} vs. 10^{-5} , warmup 20% vs. 10%, 10 vs. 5 epochs) to compensate for its disrupted initialization; scopes are screened on one seed (MLP-in at full budget, the others at a reduced budget, so the comparison is indicative) and the selected scope is validated at full budget on three seeds.

Attacks and controls (Pythia). Robustness is assessed with causal-LM analogues of the T5 attacks (HotFlip, $n = 200$, up to 10 flips, success threshold 15%; ten-token UATs, three restarts, 50 iterations, optimized on 80 and evaluated on 120 texts), with attack effect measured as the relative increase in mean per-token NLL of the perturbed sequence; since the inserted or substituted tokens are themselves prediction targets, absolute figures in this track are inflated by their low likelihood, although comparisons between the two models remain valid.

To distinguish robustness from gradient masking we add three controls: *substitution transfer* (substitutions found on one model replayed against the other; likewise for triggers), a *random-substitution control* (the same number of substitutions at random positions, with no model or gradient involved), and a *gradient-free query attack* (greedy search over 20 candidates at each of 10 random positions, scored by the target model’s own loss).

5 MECHANISMS OF GRADIENT ATTENUATION IN MONOTONE FEED-FORWARD NETWORKS

We propose a candidate mechanism for the empirical robustness gains: rather than claiming global guarantees, we analyze how monotonicity alters the local gradient structure of feed-forward sublayers in a way that can reduce the effectiveness of gradient-based attacks such as HotFlip. The analysis is motivated by monotone dynamical systems, in which cooperative interactions restrict the directions along which trajectories evolve (Hirsch, 1985); a network is not such a system, but the analogy suggests that monotonicity can limit the active directions along which outputs and gradients vary (Appendix A.4). Proofs are in Appendices A.3 and A.4.

Consider an A -monotone update $F(h) = h + A^\dagger g(Ah)$ with $s := Ah$ and the induced semantic refinement map $T(s) := AF(h) = s + g(s)$. In our experiments $A = I$ and the FFN branch acts on the normalized state $u = \text{LN}(h)$ (Remark A.1), so we take $s := u$ as the branch input and g as the branch itself; because the block is pre-normalized, the Jacobian of the sublayer map $h \mapsto h + g(\text{LN}(h))$ is $I_d + J_g(\text{LN}(h)) J_{\text{LN}}(h)$ rather than $J_T = I + J_g$, so the correction term below acquires the unconstrained factor $J_{\text{LN}}(h)$; all statements below concern the branch factor J_g .

Lemma 5.1 (Non-negativity of the Semantic Jacobian). *Let g be an MLP with elementwise non-decreasing activation σ and elementwise nonnegative weight matrices. At every s at which g is differentiable (all s for $\sigma \in C^1$; almost every s for ReLU), $J_T(s) = I + J_g(s)$ has non-negative entries (the same holds for every element of the Clarke generalized Jacobian and for the matrix returned by automatic differentiation; Appendix A.4).*

For a two-layer $g(s) = W_2\sigma(W_1s + b_1) + b_2$, $J_g(s) = W_2 \text{diag}(\sigma'(W_1s + b_1)) W_1 \geq 0$ elementwise (deeper g gives a product of nonnegative matrices): increasing any semantic coordinate cannot decrease any output coordinate through T , so sign cancellations are ruled out *within semantic space*. Writing ℓ for the loss as a function of the sublayer output and $\nabla_{s'}\mathcal{L} := (A^\dagger)^\top \nabla \ell$ for the downstream gradient in semantic coordinates, the chain rule gives $\nabla_h \ell(F(h)) = \nabla \ell + A^\top J_g^\top \nabla_{s'}\mathcal{L}$: the residual path is unaffected by the constraint, and the semantic-space gradient $\nabla_s \mathcal{L}(T(s)) := J_T^\top \nabla_{s'}\mathcal{L} = \nabla_{s'}\mathcal{L} + J_g^\top \nabla_{s'}\mathcal{L}$ adds a correction $J_g^\top \nabla_{s'}\mathcal{L}$ whose matrix is nonnegative (Lemma 5.1), which rules out sign cancellation inside the correction but does not by itself reduce its magnitude ($J_T \geq I$).

Saturation-induced attenuation. A hidden unit whose activation is flat at its current pre-activation contributes nothing to the gradient flowing through it; the assumption below fixes what “flat” means for our activations, and the lemma quantifies the contribution of nearly flat units.

Assumption 5.2 (Saturating Activations). The activation σ inside g has bounded derivative and saturates in the negative direction: $|\sigma'(z)| \rightarrow 0$ as $z \rightarrow -\infty$, with $|\sigma'|$ non-decreasing on a tail $(-\infty, z^*]$. This holds for ReLU ($\sigma'(z) = 0$ for $z < 0$, $z^* = 0$), softplus and the logistic function ($z^* \leq 0$), and GELU ($z^* = -\sqrt{2}$); ReLU, softplus, and GELU do not saturate in the positive direction ($\sigma' \rightarrow 1$), and only negative-tail saturation is used below.

Lemma 5.3 (Gradient Attenuation under Saturation). *Let $T(s) = s + g(s)$ with $g(s) = W_2\sigma(W_1s + b_1) + b_2$, $W_1 \in \mathbb{R}^{m \times p}$ and $W_2 \in \mathbb{R}^{p \times m}$ elementwise nonnegative, and $z_j(s) := (W_1s + b_1)_j$. Wherever σ is differentiable at $z(s)$, $J_g(s) = \sum_{j=1}^m \sigma'(z_j(s)) W_2[:, j] W_1[j, :]$ (one rank-one term per hidden unit). If $S \subseteq \{1, \dots, m\}$ satisfies $\max_{j \in S} |\sigma'(z_j(s))| \leq \varepsilon$ and $\|\nabla_{s'}\mathcal{L}\|_2 \leq G$, then*

$$\left\| \nabla_s \mathcal{L}(T(s)) - \left(I + \sum_{j \notin S} \sigma'(z_j(s)) W_2[:, j] W_1[j, :] \right)^\top \nabla_{s'}\mathcal{L} \right\|_2 \leq \varepsilon G \sum_{j \in S} \|W_2[:, j]\|_2 \|W_1[j, :]\|_2.$$

In particular, if σ satisfies Assumption 5.2, then along any sequence on which the pre-activations of the units in S diverge to $-\infty$ while $\nabla_{s'}\mathcal{L}$ remains bounded, the contribution of those units to J_g , and hence to $\nabla_s \mathcal{L}(T(s))$, vanishes.

Saturation thus locally suppresses the feed-forward branch’s own contribution to the backpropagated gradient, driving the sublayer Jacobian toward the identity of the residual path. Nonnegativity is not needed for the bound; its role is to prevent cancellation among the remaining active units, which makes the suppression a genuine attenuation of $\|\nabla_s \mathcal{L}(T(s))\|$ toward $\|\nabla_{s'}\mathcal{L}\|$ when $\sigma' \geq 0$ and the downstream gradient is sign-coherent (exactly for ReLU; for GELU only the norm bound is available; Appendix A.4).

Saturation is moreover one-sidedly stable: a first-layer unit with $z_j(s) \leq z^*$ and $|\sigma'(z_j(s))| \leq \varepsilon$ stays in that state under any monotone decrease $\delta \leq 0$ of the semantic coordinates, since $W_1 \geq 0$ can only lower its pre-activation and $|\sigma'|$ is non-decreasing on the tail $(-\infty, z^*]$ by Assumption 5.2 (Lemma A.2, Appendix A.4); a monotone increase can re-activate it.

Implications and scope. The analysis is conditional: it does not show that the constraint *produces* saturation (an empirical question we do not test), but that *if* units of g are saturated, their contribution to the semantic gradient is small (Lemma 5.3), this state is stable under monotone decreases (Lemma A.2), and the remaining active units cannot cancel one another (Lemma 5.1; exact for ReLU, up to GELU’s small negative-derivative region), which reduces the number and variability of directions through which a gradient-based attacker such as HotFlip can move the sublayer output. The mechanism is local to one feed-forward branch, since the token-embedding gradient also flows through unconstrained residual, attention, and normalization paths, so the gradient-free controls of Section 6.4 (Pythia track) are needed to establish that the observed robustness is not an artifact of gradient-based optimization.

Table 1: ROUGE $F_1 \times 100$ on the full CNN/DailyMail test set ($n = 11,490$), mean $\pm$ s.d. over five seeds (within-seed 95% bootstrap confidence intervals, CIs, are within ± 0.25); ROUGE-L is the summary-level variant; the standard model is not fine-tuned.

Model	ROUGE-1	ROUGE-2	ROUGE-L
Standard	38.6	17.1	33.2
Baseline	39.8 ± 0.1	16.9 ± 0.0	33.2 ± 0.1
Monotonic	37.9 ± 0.2	15.2 ± 0.2	31.5 ± 0.2

Table 2: HotFlip on CNN/DailyMail ($n = 200$, five flips, training seed 2024): mean relative loss increase, success rate (degradation above 10%), and mean absolute loss increase (nats).

Model	Avg. Deg.	Success Rate	Δ Loss
Standard	19.9%	68.5%	+0.40
Baseline	15.1%	59.5%	+0.35
Monotonic	4.9%	17.0%	+0.14

6 RESULTS

T5 numbers are means $\pm$ s.d. over five training seeds (Appendix A.6) on full test sets unless stated otherwise; additional results are in Appendix A.9.

6.1 TRAINING DYNAMICS AND SUMMARIZATION QUALITY

The monotonic model starts with a substantially higher first-epoch training loss than the baseline (4.99 vs. 2.94; Table 5 in Appendix A.9), reflecting the replacement of the pretrained FFN weights by their magnitudes, but converges reliably and almost identically across seeds (s.d. at most 0.02 in training and 0.01 in validation loss at every epoch): validation loss falls from 2.92 after epoch 1 to 2.51 at epoch 7, leaving a persistent gap of 0.30 (2.51 vs. 2.21). The ablation in Appendix A.10 attributes this gap to the loss of the pretrained sign pattern at initialization rather than to the constraint itself: an unconstrained model initialized at $|W_{\text{pre}}|$ reaches the same validation loss as the monotone model, whereas a sign-preserving softplus reparameterization comes within about 0.06 of the baseline. Because nonnegative weights cannot retain the pretrained sign pattern, we treat this initialization cost as intrinsic to the constraint.

On the full CNN/DailyMail test set (Table 1) the monotonic model achieves ROUGE-L 31.5 ± 0.2 vs. 33.2 ± 0.1 for the baseline, ROUGE-1 37.9 vs. 39.8, and ROUGE-2 15.2 vs. 16.9 (relative decreases of 4.9, 5.0, and 10.0 percent on unrounded means); per-example paired t -tests are highly significant on every metric and seed (Bonferroni-corrected $p < 10^{-57}$) but effect sizes are small (Cohen’s d 0.15–0.24), so the cost is systematic yet modest. On the full XSUM test set (11,333 articles; Table 6, Appendix A.9) the gap is smaller: ROUGE-L 12.8 ± 0.0 vs. 13.1 ± 0.0 (2.1 percent; ROUGE-1 3.7 percent; $p < 1.4 \times 10^{-6}$, $d \leq 0.14$). The trade-off is thus consistent across news articles and single-sentence summaries, and output lengths and brevity penalties are close for the two fine-tuned models (Appendix A.9).

6.2 ADVERSARIAL ROBUSTNESS

HotFlip. Table 2 reports results on 200 held-out CNN/DailyMail examples for the seed-2024 checkpoints (a seed-42 evaluation on 100 examples agrees; Appendix A.9). The monotonic model incurs an average degradation of 4.9 percent vs. 15.1 percent for the baseline and 19.9 percent for the standard model, and only 17.0 percent of attacks succeed against it vs. 59.5 and 68.5 percent: a 71 percent relative reduction in success rate and a 67 percent reduction in degradation. Because relative degradation is normalized by each model’s own clean loss (3.39 vs. 2.77 nats), we also report the absolute increase, 0.14 nats vs. 0.35 (baseline) and 0.40 (standard), so the gap is not an artifact of normalization. Independent-samples t -tests confirm the baseline–monotonic difference ($t = 9.04$, $p = 6.9 \times 10^{-18}$; vs. standard $t = 10.49$, $p = 6.5 \times 10^{-23}$) with a large effect size (Cohen’s $d = 0.91$).

Universal triggers. Universal triggers prove minimally effective against T5, with NLL increases below 1 percent for all models (four seeds): 0.33 ± 0.05 percent for the monotonic model vs. 0.56 ± 0.06 for the baseline and 0.74 ± 0.09 for standard T5, an ordering that holds on every seed; ROUGE-L changes and cross-model transfer are negligible (Appendix A.11). These effects are consistent with improved robustness but too small for strong conclusions, and given the truncation caveat of Section 4 they do not show that input-agnostic perturbations are ineffective against sequence-to-sequence models in general; the T5 robustness benefit rests on the adaptive HotFlip attack.

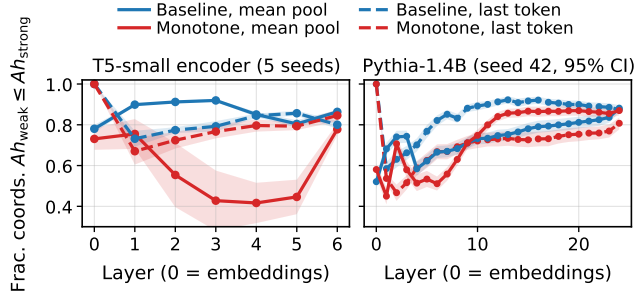


Figure 1: Order preservation along learned semantic directions versus depth. Left: T5-small encoder (mean $\pm$ s.d., five seeds). Right: Pythia-1.4B (MLP-both, seed 42; 95% bootstrap CIs, 195 held-out pairs). Layer 0 is the embedding output; its last-token value is trivially 1 since all prompts share the final token.

Table 3: Pythia-1.4B, MLP-both, full budget. PPL: held-out Pile perplexity; SR: HotFlip success rate ($n = 200$, 10 flips, 15% threshold); UAT: relative and absolute NLL increase under the trigger (base $\rightarrow$ mono).

Seed	PPL base	PPL mono	SR base $\rightarrow$ mono	UAT NLL $\uparrow$	UAT Δ NLL (nats)
42	6.72	36.01	69.0% $\rightarrow$ 43.5%	+15.5% $\rightarrow$ +8.8%	0.31 $\rightarrow$ 0.33
1337	6.72	42.00	69.0% $\rightarrow$ 39.0%	+16.4% $\rightarrow$ +8.1%	0.33 $\rightarrow$ 0.32
2024	6.72	35.03	69.0% $\rightarrow$ 45.0%	+16.5% $\rightarrow$ +15.0%	0.33 $\rightarrow$ 0.56
Mean	6.72	37.68	69.0% $\rightarrow$ 42.5%	+16.1% $\rightarrow$ +10.6%	0.32 $\rightarrow$ 0.40

6.3 ORDER PRESERVATION ALONG SEMANTIC DIRECTIONS

Whether the per-sublayer, coordinatewise constraint preserves the *semantic* orderings that motivate it (Section 1) end to end is an empirical question, because attention, normalization, and residual mixing are unconstrained. Figure 1 reports the probe diagnostic of Section 4: on T5 the result is negative at intermediate depth and neutral at the output, and on Pythia-1.4B negative at every depth. With mean pooling the baseline T5 encoder keeps 0.78–0.92 of probe coordinates correctly ordered at every layer, whereas the monotone encoder falls to 0.42–0.45 at layers 3–5 before recovering to 0.78 at the output (five-seed means; the dip is present in every seed); with last-token pooling the two are within 0.07 at every layer and the monotone model is slightly higher at the output (0.85 vs. 0.80). On Pythia-1.4B (seed 42, last-token pooling) the baseline reaches 0.92 by layer 13 while the monotone model plateaus at 0.73–0.81. Coordinatewise nonnegativity of the FFN weights therefore does not by itself make the network more order-preserving along these semantic directions; the robustness gains are more plausibly attributed to the local sensitivity properties of the constrained sublayers (Section 5) than to end-to-end semantic monotonicity.

6.4 GENERALIZATION TO DECODER-ONLY FOUNDATION MODELS

Constraint scope. In the screening (Table 7, Appendix A.9), constraining only the MLP input projection fails: perplexity rises 3.65-fold yet the HotFlip success rate *rises* from 71.5% to 83.5%, suggesting that gradient signal routes around the constrained matrix through the unconstrained output projection and attention. Constraining both MLP projections (MLP-both, the T5 scope) reduces the success rate by 30.5 points; constraining the attention output projection instead of W_{out} (MLP-in + Attn-out) reduces it by 39.5 points at about double the perplexity penalty ($12.49\times$ vs. $6.80\times$), a diminishing return, so we select MLP-both.

Multi-seed validation. Table 3 reports MLP-both at full budget on three seeds. The HotFlip success rate falls from 69.0% to 39.0–45.0% (mean 42.5%), a 26.5-point (38.4% relative) reduction. Universal triggers, unlike on T5, substantially damage the decoder-only baseline (NLL +16.1%) and the monotone models are less affected in relative terms (+10.6%), but their clean NLL is roughly 1.9 times higher and the absolute increase is not smaller (0.40 ± 0.14 vs. 0.32 ± 0.01 nats): monotonicity reduces the relative, but not the absolute, effect of triggers.

Table 4: Transfer and gradient-free controls on Pythia-1.4B (MLP-both, full budget; mean $\pm$ s.d. over three seeds; 10 substitutions on 200 held-out texts, query attack on 50). Deg.: mean relative loss increase; SR: success rate (15% threshold); Δ NLL: absolute increase (nats). HotFlip rows re-run Table 3’s attack inside the transfer pipeline.

Attack	Crafted on	Evaluated on baseline			Evaluated on monotone		
		Deg.	SR	Δ NLL	Deg.	SR	Δ NLL
HotFlip	baseline	$55.2 \pm 0.4\%$	$68.7 \pm 0.3\%$	0.82	$15.4 \pm 1.3\%$	$25.8 \pm 1.6\%$	0.57
HotFlip	monotone	$65.0 \pm 0.9\%$	$72.8 \pm 1.8\%$	0.94	$20.5 \pm 1.6\%$	$42.5 \pm 3.1\%$	0.77
Random substitutions	—	$64.6 \pm 0.3\%$	$73.0 \pm 1.0\%$	0.91	$17.9 \pm 1.5\%$	$36.5 \pm 2.3\%$	0.66
Query (gradient-free)	target	$112.3 \pm 4.0\%$	$77.3 \pm 1.2\%$	1.39	$33.6 \pm 2.9\%$	$50.0 \pm 3.5\%$	1.17

Transfer and gradient-free controls. Table 4 reports the controls of Section 4, which address the concern that a constrained model merely masks gradients (Athalye et al., 2018; Carlini et al., 2019; Tramèr et al., 2020). First, the monotone model’s advantage persists under every gradient-free attack (random substitutions degrade the baseline by 64.6 percent vs. 17.9 percent for the monotone model; the query attack by 112 vs. 34 percent); since neither uses gradients, the gap cannot be gradient masking. Second, attacks do not transfer symmetrically: substitutions crafted on the baseline degrade the monotone model by only 15.4 percent, less than random ones, whereas substitutions crafted on the monotone model degrade the baseline by 65.0 percent, as much as random ones (triggers behave the same way; Appendix A.11). Third, absolute gaps are narrower (random 0.66 vs. 0.91 nats; query 1.17 vs. 1.39; HotFlip 0.77 vs. 0.82) because the monotone model’s clean NLL is roughly 1.9 times higher: it is less sensitive to every substitution type tested, but part of its relative advantage reflects its higher clean loss, a caveat far weaker for T5 (absolute gap 2.5-fold).

The quality–robustness trade-off is architecture-dependent. Monotonicity is not universally “free”: held-out perplexity of the monotone Pythia model is $5.61\times$ that of its baseline even after full-budget recovery training, whereas the T5 monotone model’s validation perplexity is only about $1.35\times$ its baseline’s with ROUGE-L within 5%. We conjecture an architectural explanation: in a decoder-only block the constrained FFN must absorb a larger share of the block’s transformation, whereas T5’s encoder–decoder bottleneck and cross-attention carry more of it unconstrained and also restrict the routes by which perturbations bypass constrained sublayers.

7 DISCUSSION AND CONCLUSION

This work provides empirical evidence, and a conditional theoretical mechanism, that monotonicity is a promising architectural bias for language-model robustness: enforcing it in feed-forward sublayers consistently reduces vulnerability to token-level attacks while largely preserving task performance in encoder–decoder models. The gradient-free Pythia-1.4B controls show that the advantage is not confined to gradient-based attacks, so the mechanism of Section 5 is one facet of a more general reduction in sensitivity to token-level perturbations rather than gradient masking. The order-preservation diagnostic shows that per-sublayer, coordinatewise monotonicity does not itself make the network more order-preserving along learned semantic directions: that ordering (Section 1) is the design principle, whereas the experiments support a local sensitivity reduction in the constrained sublayers; imposing the constraint in a learned semantic basis A is a natural next step (see Appendix A.12).

Our study is limited to modest model scales, provides no formal certificates, and leaves open whether the robustness gain, like the clean-quality cost, is attributable to the initialization rather than the constraint (Appendix A.10). Consistent improvements across two Transformer families nevertheless suggest that monotonicity is a principled architectural prior for robust language models at an architecture-dependent cost in clean performance.

AI USE STATEMENT

Generative AI tools (Anthropic’s Claude, via Claude Code) were used in this work to help develop and debug the experiment pipeline that implements the method and attacks, to compute, from the raw per-seed outputs, the aggregate statistics reported in the tables and to draft their interpretation, and to check and correct the statements and proofs of Section 5 and Appendix A.3; the corrected

statements were re-derived and verified by the authors, and every reported number was recomputed by the authors from the raw outputs. Generative AI tools were not used to generate synthetic data or to propose the hypotheses or the research methodology, and no experiment was designed, run, or altered by an AI tool; the remaining required-disclosure categories of the ICLR policy are not applicable to this work. AI tools were also used to draft and edit parts of the text for readability, to identify and verify related literature and format references, and to produce Figure 1 from the experimental outputs. All AI-assisted code was tested by the authors, all AI-suggested references were checked against their primary sources, and all AI-assisted text and mathematics were reviewed by the authors. We take responsibility for the final content of this work, including text, claims, and artifacts produced with the aid of generative AI.

ETHICS STATEMENT

This work explores architectural constraints that improve the robustness of language models to adversarial manipulation. By studying monotonicity as a structural inductive bias, our approach contributes to the development of language models whose behavior is more predictable and analyzable, which is particularly relevant for safety-critical and decision-support applications. All experiments use publicly available pretrained models (T5-small, Pythia-1.4B) and public datasets; no human subjects or private data are involved. The attacks studied are standard, previously published token-level attacks evaluated on summarization and language-modeling loss, and do not produce new capabilities for misuse.

REPRODUCIBILITY STATEMENT

Sections 3 and 4 state the method as implemented (the coordinatewise instantiation, the softplus reparameterization, and the initialization), and Appendix A.5, Appendix A.6, and Appendix A.7 give the complete training, evaluation, attack, and hardware settings for both tracks, including the seeds, the statistical tests, and the precision used at each stage. All theoretical statements are proved in Appendices A.2, A.3, and A.4, with their assumptions stated explicitly. Every multi-seed entry in the tables is a mean (and, where shown, standard deviation) over the per-seed result files, as described in Appendix A.6; Tables 2, 7, and 8 report single seeds, as stated in their captions. The code for the full pipeline (training, attacks, controls, and the order-preservation probe) and the per-seed result files will be released with the camera-ready version.

REFERENCES

- Brandon Amos and J. Zico Kolter. Optnet: Differentiable optimization as a layer in neural networks. In *International Conference on Machine Learning (ICML)*, 2017.
- David Angeli and Eduardo D. Sontag. Monotone control systems. *IEEE Transactions on Automatic Control*, 48(10):1684–1698, 2003. doi: 10.1109/TAC.2003.817920.
- Cem Anil, Esin Durmus, Nina Panickssery, Mrinank Sharma, Joe Benton, Sandipan Kundu, Joshua Batson, Meg Tong, Jesse Mu, Daniel Ford, et al. Many-shot jailbreaking. *Advances in Neural Information Processing Systems*, 37:129696–129742, 2024.
- Anish Athalye, Nicholas Carlini, and David Wagner. Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018. arXiv:1802.00420.
- Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 2397–2430. PMLR, 2023.
- Bochuan Cao, Yuanpu Cao, Lu Lin, and Jinghui Chen. Defending against alignment-breaking attacks via robustly aligned llm. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10542–10560, 2024.

- Nicholas Carlini, Anish Athalye, Nicolas Papernot, Wieland Brendel, Jonas Rauber, Dimitris Tsipras, Ian Goodfellow, Aleksander Madry, and Alexey Kurakin. On evaluating adversarial robustness. *arXiv preprint arXiv:1902.06705*, 2019.
- Minhao Cheng, Jinfeng Yi, Pin-Yu Chen, Huan Zhang, and Cho-Jui Hsieh. Seq2Sick: Evaluating the robustness of sequence-to-sequence models with adversarial examples. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 3601–3608, 2020. AAAI-20 Technical Tracks 4.
- Jan Chorowski and Jacek M. Zurada. Learning understandable neural networks with nonnegative weight constraints. *IEEE Transactions on Neural Networks and Learning Systems*, 26(1):62–69, 2015. doi: 10.1109/TNNLS.2014.2310059.
- Hennie A. M. Daniëls and Marina V. Velikova. Monotone and partially monotone neural networks. *IEEE Transactions on Neural Networks*, 21(6):906–917, 2010. doi: 10.1109/TNN.2010.2044803.
- Yue Deng, Wenxuan Zhang, Sinno Jialin Pan, and Lidong Bing. Multilingual jailbreak challenges in large language models. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.06474>.
- Javid Ebrahimi, Anyi Rao, Daniel Lowd, and Dejing Dou. HotFlip: White-box adversarial examples for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 31–36. Association for Computational Linguistics, 2018.
- Hamid Reza Feyzmahdavian, Bart Besselink, and Mikael Johansson. Stability analysis of monotone systems via max-separable Lyapunov functions. *IEEE Transactions on Automatic Control*, 63(3): 643–656, 2018. doi: 10.1109/TAC.2017.2727282.
- Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The Pile: An 800GB dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.
- Chuan Guo, Alexandre Sablayrolles, Hervé Jégou, and Douwe Kiela. Gradient-based adversarial attacks against text transformers. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 5747–5757, 2021.
- Maya Gupta, Andrew Cotter, Jan Pfeifer, Konstantin Voevodski, Kevin Canini, Alexander Mangylov, Wojciech Moczydlowski, and Alexander van Esbroeck. Monotonic calibrated interpolated look-up tables. *Journal of Machine Learning Research*, 17(109):1–47, 2016. URL <https://jmlr.org/papers/v17/15-243.html>.
- Morris W Hirsch. Systems of differential equations that are competitive or cooperative ii: Convergence almost everywhere. *SIAM Journal on Mathematical Analysis*, 16(3):423–439, 1985.
- Morris W. Hirsch and Hal Smith. Monotone dynamical systems. In *Handbook of Differential Equations: Ordinary Differential Equations*, volume 2, chapter 4, pp. 239–357. Elsevier, 2006. doi: 10.1016/S1874-5725(05)80006-9.
- Nikolaus H. R. Howe, Ian R. McKenzie, Oskar John Hollinsworth, Michał Zając, Tom Tseng, Aaron David Tucker, Pierre-Luc Bacon, and Adam Gleave. Scaling trends in language model robustness. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 24080–24138. PMLR, 2025. arXiv:2407.18213.
- Christian Igel. Smooth min-max monotonic networks. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*. PMLR, 2024. arXiv:2306.01147.
- Íñigo Íncer Romeo, Michael Theodorides, Sadia Afroz, and David Wagner. Adversarially robust malware detection using monotonic classification. In *Proceedings of the Fourth ACM International Workshop on Security and Privacy Analytics (IWSPA)*, 2018. doi: 10.1145/3180445.3180449.

- Hiroshi Ito, Björn S Rüffer, and Anders Rantzer. Max-and sum-separable lyapunov functions for monotone systems and their level sets. In *53rd IEEE Conference on Decision and Control*, pp. 2371–2377. IEEE, 2014.
- Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping-yeh Chiang, Micah Goldblum, Aniruddha Saha, Jonas Geiping, and Tom Goldstein. Baseline defenses for adversarial attacks against aligned language models. *arXiv preprint arXiv:2309.00614*, 2023.
- Robin Jia, Aditi Raghunathan, Kerem Göksel, and Percy Liang. Certified robustness to adversarial word substitutions. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 4129–4142, 2019.
- Di Jin, Zhijing Jin, Joey Tianyi Zhou, and Peter Szolovits. Is BERT really robust? A strong baseline for natural language attack on text classification and entailment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 8018–8025, 2020. AAAI-20 Technical Tracks 5.
- Stasys Jukna. *Boolean Function Complexity: Advances and Frontiers*, volume 27 of *Algorithms and Combinatorics*. Springer, Berlin, Heidelberg, 2012. doi: 10.1007/978-3-642-24508-4.
- David Khachaturov and Robert Mullins. Adversarial suffix filtering: a defense pipeline for llms. *arXiv preprint arXiv:2505.09602*, 2025.
- Hyunjik Kim, George Papamakarios, and Andriy Mnih. The Lipschitz constant of self-attention. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pp. 5562–5571. PMLR, 2021.
- Ouail Kitouni, Niklas Nolte, and Mike Williams. Robust and provably monotonic networks. *Machine Learning: Science and Technology*, 4, 2023. doi: 10.1088/2632-2153/aced80. arXiv:2112.00038.
- Miles Q Li and Benjamin CM Fung. Security concerns for large language models: A survey. *Journal of Information Security and Applications*, 95:104284, 2025.
- Xingchao Liu, Xing Han, Na Zhang, and Qiang Liu. Certified monotonic neural networks. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. arXiv:2011.10219.
- Anthony N Michel, Ling Hou, and Derong Liu. *Stability of dynamical systems: On the role of monotonic and non-monotonic Lyapunov functions*. Springer, 2015.
- Dan Mikulincer and Daniel Reichman. Size and depth of monotone neural networks: Interpolation and approximation. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022. arXiv:2207.05275.
- Laker Newhouse, R Preston Hess, Franz Cesista, Andrii Zahorodnii, Jeremy Bernstein, and Phillip Isola. Training transformers with enforced lipschitz constants. *arXiv preprint arXiv:2507.13338*, 2025.
- An-Phi Nguyen, Dana Lea Moreno, Nicolas Le-Bel, and María Rodríguez Martínez. Mononet: enhancing interpretability in neural networks via monotonic features. *Bioinformatics advances*, 3(1):vbad016, 2023.
- Niklas Nolte, Ouail Kitouni, and Mike Williams. Expressive monotonic neural networks. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. arXiv:2307.07512.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.
- Anders Rantzer and Bo Bernhardsson. Control of convex-monotone systems. In *53rd IEEE Conference on Decision and Control*, pp. 2378–2383. IEEE, 2014.

- Alexander Robey, Eric Wong, Hamed Hassani, and George J. Pappas. SmoothLLM: Defending large language models against jailbreaking attacks. *arXiv preprint arXiv:2310.03684*, 2023. URL <https://arxiv.org/abs/2310.03684>.
- Davor Runje and Sharath M Shankaranarayana. Constrained monotonic neural networks. In *International Conference on Machine Learning*, pp. 29338–29353. PMLR, 2023.
- Davide Sartor, Alberto Sinigaglia, and Gian Antonio Susto. Advancing constrained monotonic neural networks: Achieving universal approximation beyond bounded activations. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 52995–53018. PMLR, 2025. arXiv:2505.02537.
- Md. Faiyaz Abdullah Sayeedi, Maaz Bin Hossain, Md. Kamrul Hassan, Sabrina Afrin, Molla Md. Sabit Hossain, and Md. Shohrab Hossain. JailbreakTracer: Explainable detection of jailbreaking prompts in LLMs using synthetic data generation. *IEEE Access*, 13:123708–123723, 2025. doi: 10.1109/ACCESS.2025.3579996.
- Taylor Shin, Yasaman Razeghi, Robert L. Logan IV, Eric Wallace, and Sameer Singh. AutoPrompt: Eliciting knowledge from language models with automatically generated prompts. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 4222–4235, 2020.
- Joseph Sill. Monotonic networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1998.
- Aishwarya Sivaraman, Golnoosh Farnadi, Todd Millstein, and Guy Van den Broeck. Counterexample-guided learning of monotonic neural networks. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/8ab70731b1553f17c11a3bbc87e0b605-Abstract.html>.
- Hal L Smith. *Monotone dynamical systems: an introduction to the theory of competitive and cooperative systems*, volume 41 of *Mathematical Surveys and Monographs*. American Mathematical Society, 1995.
- Jingtong Su, Julia Kempe, and Karen Ullrich. Mission impossible: A statistical perspective on jailbreaking llms. *Advances in Neural Information Processing Systems*, 37:38267–38306, 2024.
- Florian Tramèr, Nicholas Carlini, Wieland Brendel, and Aleksander Madry. On adaptive attacks to adversarial example defenses. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. arXiv:2002.08347.
- Eric Wallace, Shi Feng, Nikhil Kandpal, Matt Gardner, and Sameer Singh. Universal adversarial triggers for attacking and analyzing NLP. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 2153–2162. Association for Computational Linguistics, 2019.
- Boxin Wang, Chejian Xu, Shuohang Wang, Zhe Gan, Yu Cheng, Jianfeng Gao, Ahmed Hassan Awadallah, and Bo Li. Adversarial GLUE: A multi-task benchmark for robustness evaluation of language models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS Datasets and Benchmarks)*, 2021. URL <https://arxiv.org/abs/2111.02840>.
- Yiwei Wang, Muhao Chen, Nanyun Peng, and Kai-Wei Chang. Vulnerability of large language models to output prefix jailbreaks: Impact of positions on safety. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 3939–3952, 2025.
- Antoine Wehenkel and Gilles Louppe. Unconstrained monotonic neural networks. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019. arXiv:1908.05164.
- Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How does LLM safety training fail? In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. arXiv:2307.02483.

- Ezra Winston and J. Zico Kolter. Monotone operator equilibrium networks. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. arXiv:2006.08591.
- Seungil You, David Ding, Kevin Canini, Jan Pfeifer, and Maya Gupta. Deep lattice networks and partial monotonic functions. In *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/464d828b85b0bed98e80ade0a5c43b0f-Abstract.html>.
- Sicheng Zhu, Ruiyi Zhang, Bang An, Gang Wu, Joe Barrow, Zichao Wang, Furong Huang, Ani Nenkova, and Tong Sun. AutoDAN: Interpretable gradient-based adversarial attacks on large language models. *arXiv preprint arXiv:2310.15140*, 2023. URL <https://arxiv.org/abs/2310.15140>.
- Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023. URL <https://arxiv.org/abs/2307.15043>.
- Andy Zou, Long Phan, Justin Wang, Derek Duenas, Maxwell Lin, Maksym Andriushchenko, Rowan Wang, Zico Kolter, Matt Fredrikson, and Dan Hendrycks. Improving alignment and robustness with circuit breakers. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, pp. 83345–83373, 2024. arXiv:2406.04313.

A APPENDIX

A.1 EXTENDED RELATED WORK

Vulnerabilities of Language Models. Large language models are highly susceptible to adversarial manipulations that bypass alignment and safety mechanisms. Prior work has documented jailbreak prompts and transferable adversarial suffixes that generalize across models and settings, including aligned systems (Zou et al., 2023). Subsequent studies identify additional failure modes, such as position-sensitive jailbreaks (Wang et al., 2025), interpretable attacks that evade perplexity-based defenses (Zhu et al., 2023), and many-shot jailbreaks whose effectiveness increases with larger context windows (Anil et al., 2024). Robustness benchmarks further show that models perform poorly under systematic adversarial evaluation (Wang et al., 2021), and that safety mechanisms often fail in multilingual or low-resource settings (Deng et al., 2024). Prompt injection attacks, which exploit the lack of separation between instruction and context, are now recognized as a fundamental vulnerability, alongside broader threats to inference and training-time surveyed in recent security analyses (Li & Fung, 2025). At the token level, the attacks we use descend from a well-developed lineage: gradient-guided substitutions (Ebrahimi et al., 2018), universal triggers (Wallace et al., 2019), and their prompt-search descendants (Shin et al., 2020); fully gradient-based relaxations for Transformers (Guo et al., 2021); gradient-free, query-based word substitutions (Jin et al., 2020); and attacks designed specifically for sequence-to-sequence models (Cheng et al., 2020).

Defenses and Robustness Enhancements. Proposed defenses include smoothing-based methods with provable jailbreak guarantees (Robey et al., 2023), prompt-level classifiers trained on synthetic jailbreak data (Sayeedi et al., 2025), and adversarial training approaches whose effectiveness depends on model scale and threat model (Howe et al., 2025). Adversarial prompts also transfer across models, so that a defense evaluated against a single attacker can be circumvented by suffixes optimized elsewhere (Zou et al., 2023). Simple input-side baselines such as perplexity filtering and paraphrasing are surprisingly competitive against optimization-based attacks (Jain et al., 2023), representation-level interventions reroute the internal activations responsible for harmful outputs (Zou et al., 2024), and certified training bounds the effect of word substitutions for small classifiers (Jia et al., 2019). Despite these advances, most defenses remain reactive and training-dependent. A recurring lesson from the vision literature is that apparent robustness to gradient-based attacks can be an artifact of obfuscated gradients, so that any architectural defense must be evaluated with gradient-free and adaptive attacks as well (Athalye et al., 2018; Carlini et al., 2019; Tramèr et al., 2020); the random-substitution and query-based controls of Section 6.4 follow this guidance.

Monotonic Neural Networks and Structural Constraints. Monotonicity has long been studied as a structural inductive bias for interpretability and robustness. Classical results show that feed-forward networks with non-negative weights and sigmoidal activations (Daniëls & Velikova, 2010), as well as min-max networks of sign-constrained linear units (Sill, 1998), are universal approximators of monotone functions, establishing that monotonicity need not limit expressivity. Subsequent work proposed practical monotone and partially monotone architectures, including lattice-based models, enabling scalable learning under monotonic constraints in structured domains (Gupta et al., 2016; You et al., 2017). More recent work certifies monotonicity of piecewise-linear networks by mixed-integer linear programming (Liu et al., 2020) or counterexample-guided repair (Sivaraman et al., 2020). Alternatives to sign-constrained weights include integrating a positive derivative network (Wehenkel & Louppe, 2019), smooth min-max constructions (Igel, 2024), and Lipschitz-bounded residual architectures that are monotone by construction and simultaneously certify robustness (Nolte et al., 2023; Kitouni et al., 2023); the price of the nonnegative-parameter inductive bias in depth and width has been characterized theoretically (Mikulincer & Reichman, 2022), and nonnegative weight constraints have also been used for interpretability in multilayer perceptrons (Chorowski & Zurada, 2015). The observation that monotone classifiers are structurally resistant to a class of evasion attacks predates the LLM era (İncir Romeo et al., 2018). In control theory, monotone (cooperative) systems admit order-preserving flows with strong convergence properties (Hirsch & Smith, 2006; Angeli & Sontag, 2003), and structural order can, under suitable conditions, reduce verification to extremal cases (Feyzmahdavian et al., 2018), an intuition we draw on in Section 5; monotone operator theory has separately been used to guarantee well-posedness of implicit-depth networks (Winston & Kolter, 2020). More broadly, architectural constraints and structured layers have been shown to improve robustness and stability without prohibitive performance loss (Amos & Kolter, 2017), and controlling the Lipschitz constant of attention has been proposed as a structural

route to more stable Transformers (Kim et al., 2021; Newhouse et al., 2025). Our work builds on this line by showing that selectively enforced monotonicity can scale to Transformer-based language models and yield tangible robustness gains.

A.2 ADDITIONAL DETAILS ON THE MONOTONE TRANSFORMER

Proof of Proposition 3.3. If $Ah \leq Ah'$, monotonicity of g gives $g(Ah) \leq g(Ah')$, and

$$\begin{aligned} AF(h) &= Ah + AA^\dagger g(Ah) \\ &= Ah + g(Ah) \\ &\leq Ah' + g(Ah') \\ &= AF(h'). \end{aligned}$$

□

Note that $\preceq$ is a preorder rather than a partial order whenever $p < d$: $h \preceq h' \preceq h$ holds exactly when $h - h' \in \ker(A)$, and such pairs are indistinguishable to the constraint; when $A = I$, $\preceq$ is the coordinatewise partial order.

Remark A.1 (Placement inside a pre-normalized block). In the T5 and GPT-NeoX blocks we instantiate (with $A = I$), the FFN branch receives the normalized residual state $u = \text{LN}(h)$, where LN denotes the block’s normalization layer, and the residual connection is applied outside the branch, $F(h) = h + g(\text{LN}(h))$, with $g(u) = W_o \text{ReLU}(W_i u)$ in T5 (no biases) and $g(u) = W_{\text{out}} \text{GELU}(W_{\text{in}} u + b_{\text{in}}) + b_{\text{out}}$ in GPT-NeoX. The normalization (a root-mean-square rescaling with an unconstrained gain in T5; standard layer normalization with mean centering, gain, and bias in GPT-NeoX) is not monotone, so F need not be coordinatewise monotone as a map of h . The property we enforce, and the object of the analysis in Section 5, is the monotonicity of the branch g as a map of its own input u ; the residual path is the identity and is unaffected by the constraint.

A.3 MATHEMATICAL PROOFS

A.3.1 PROOF OF LEMMA 3.2

Proof. For any $x, x' \in \mathbb{R}^n$ such that $x \leq x'$, monotonicity of f implies $f(x) \leq f(x')$. Applying monotonicity of g yields

$$g(f(x)) \leq g(f(x')),$$

which establishes the claim. □

A.3.2 PROOF OF LEMMA 5.1

Proof. Recall $T(s) = s + g(s)$, so at every point of differentiability

$$J_T(s) = \partial T(s)/\partial s = I + J_g(s).$$

For a two-layer $g(s) = W_2 \sigma(W_1 s + b_1) + b_2$ with $W_1, W_2 \geq 0$ elementwise,

$$J_g(s) = W_2 \text{diag}(\sigma'(W_1 s + b_1)) W_1.$$

Since σ is elementwise non-decreasing, $\sigma'(z) \geq 0$ wherever it exists, so $\text{diag}(\sigma'(\cdot)) \geq 0$ elementwise. Hence $J_g(s) \geq 0$ elementwise, and therefore $J_T(s) = I + J_g(s) \geq 0$ elementwise. For an L -layer g with hidden states $h_\ell = \sigma(z_\ell)$ and pre-activations $z_\ell = W_\ell h_{\ell-1} + b_\ell$ (with $h_0 = s$),

$$J_g(s) = W_L \text{diag}(\sigma'(z_{L-1})) W_{L-1} \text{diag}(\sigma'(z_{L-2})) \cdots W_2 \text{diag}(\sigma'(z_1)) W_1$$

is a product of L weight matrices and $L - 1$ diagonal matrices, each elementwise nonnegative, and is therefore elementwise nonnegative, so the conclusion is unchanged. At a point s where some pre-activation lies at a kink of σ (for ReLU, $z_j = 0$), T is locally Lipschitz whenever σ is, and its Clarke generalized Jacobian is by definition the convex hull of all limits $\lim_k J_T(s_k)$ along sequences $s_k \rightarrow s$ of points of differentiability. Each $J_T(s_k)$ is elementwise nonnegative by the product formula, and the set of elementwise nonnegative matrices is closed and convex, so every element of the

Clarke generalized Jacobian is elementwise nonnegative. Reverse-mode automatic differentiation evaluates the same product with $\sigma'(z_j)$ at a kink replaced by the framework’s convention (for ReLU in PyTorch, $\sigma'(0) = 0$); for any nonnegative convention the result is again a product of nonnegative matrices. $\square$

A.3.3 PROOF OF LEMMA 5.3

Proof. Write $D(s) := \text{diag}(\sigma'(z(s)))$. Then $J_g(s) = W_2 D(s) W_1 = \sum_{j=1}^m \sigma'(z_j(s)) W_2[:, j] W_1[j, :]$, which is the stated decomposition. With $v := \nabla_{s'} \mathcal{L}$,

$$\nabla_s \mathcal{L}(T(s)) = (I + J_g(s))^\top v = v + \sum_{j=1}^m \sigma'(z_j(s)) W_1[j, :]^\top (W_2[:, j]^\top v).$$

Splitting the sum into $j \in S$ and $j \notin S$, the difference in the statement equals $\sum_{j \in S} \sigma'(z_j(s)) W_1[j, :]^\top (W_2[:, j]^\top v)$. By the triangle and Cauchy–Schwarz inequalities its Euclidean norm is at most

$$\sum_{j \in S} |\sigma'(z_j(s))| \|W_1[j, :]\|_2 \|W_2[:, j]\|_2 \|v\|_2 \leq \varepsilon G \sum_{j \in S} \|W_2[:, j]\|_2 \|W_1[j, :]\|_2.$$

For the final claim, if $z_j(s_k) \rightarrow -\infty$ for all $j \in S$ then $\max_{j \in S} |\sigma'(z_j(s_k))| \rightarrow 0$ by Assumption 5.2, so the bound above tends to 0 as long as $\|\nabla_{s'} \mathcal{L}\|_2$ stays bounded along the sequence. $\square$

A.4 GRADIENT ATTENUATION: FULL DERIVATIONS AND DISCUSSION

This appendix reproduces the complete text of the mechanism analysis, including the chain-rule decomposition, the discussion of sign coherence, and the scope of the analysis.

We propose a candidate mechanism for the empirical robustness gains observed in monotone Transformer language models. Rather than claiming global robustness guarantees, we analyze how monotonicity constraints alter the local gradient structure of feed-forward sublayers in a way that can reduce the effectiveness of gradient-based adversarial attacks such as HotFlip.

Our analysis is motivated by classical results in the theory of monotone dynamical systems, which show that, for strongly monotone (cooperative and irreducible) flows, almost every bounded trajectory converges to the set of equilibria (Hirsch, 1985). While a feed-forward network is not a time-indexed dynamical system in this sense, and convergence of states says nothing directly about input–output sensitivities, this perspective suggests that monotonicity can limit the set of active directions along which outputs and gradients vary. We leverage this intuition to inform a local analysis of gradient behavior under monotone perturbations, focusing on how saturation and non-negative weights interact to diminish exploitable gradient directions.

Consider an A -monotone FFN update

$$F(h) := h + A^\dagger g(Ah),$$

where $A \in \mathbb{R}^{p \times d}$ is fixed, $A^\dagger$ satisfies $AA^\dagger = I_p$, and $g : \mathbb{R}^p \rightarrow \mathbb{R}^p$ is coordinatewise monotone (e.g., an MLP with elementwise non-decreasing activations and elementwise nonnegative weight matrices). Let $s := Ah \in \mathbb{R}^p$ denote semantic coordinates and define the induced semantic refinement map

$$T(s) := AF(h) = s + g(s).$$

We analyze the Jacobian of T and its implications for gradients with respect to semantic coordinates. (In general, $J_F(h)$ need not be elementwise nonnegative in $\mathbb{R}^d$ due to $A^\dagger$.) In our experiments $A = I$, and we take as s the input $u = \text{LN}(h)$ of the FFN branch of Remark A.1, with g the branch itself. Because the block is pre-normalized, the residual adds h rather than s , so T is not literally the sublayer map $h \mapsto F(h) = h + g(\text{LN}(h))$, whose Jacobian is $I_d + J_g(\text{LN}(h)) J_{\text{LN}}(h)$; for the trained models the chain-rule decomposition below therefore holds with the correction term multiplied on the left by the unconstrained factor $J_{\text{LN}}(h)^\top$. All statements that follow concern the branch factor J_g and are unaffected by this factor.

Lemma (Lemma 5.1, full statement). *Let g be an MLP with elementwise non-decreasing activation σ and elementwise nonnegative weight matrices. At every $s \in \mathbb{R}^p$ at which g is differentiable (all s if σ is continuously differentiable; Lebesgue-almost every s for ReLU), the Jacobian*

$$J_T(s) = \partial T(s)/\partial s = I + J_g(s)$$

has non-negative entries. The same holds for every element of the Clarke generalized Jacobian of T (for locally Lipschitz σ), and likewise for the matrix returned by reverse-mode automatic differentiation under any nonnegative convention for σ' at kinks.

This follows from $T(s) = s + g(s)$ and the factorization of the MLP Jacobian. For a two-layer $g(s) = W_2\sigma(W_1s + b_1) + b_2$ with $W_1, W_2 \geq 0$ elementwise,

$$J_g(s) = W_2 \text{diag}(\sigma'(W_1s + b_1)) W_1 \geq 0 \quad (\text{elementwise}),$$

hence $J_T(s) = I + J_g(s) \geq 0$ elementwise; for deeper g the Jacobian is a product of elementwise nonnegative matrices (Appendix A.3.2). Intuitively, increasing any semantic coordinate cannot decrease any output semantic coordinate through T ; sign cancellations are ruled out *within semantic space*.

Let $\ell : \mathbb{R}^d \rightarrow \mathbb{R}$ be the differentiable objective induced by the rest of the network on the sublayer output $h' = F(h)$. Since $J_F(h) = I_d + A^\top J_g(Ah) A$, the chain rule gives

$$\nabla_h \ell(F(h)) = \nabla \ell(h') + A^\top J_g(Ah)^\top (A^\dagger)^\top \nabla \ell(h').$$

The first term is the residual path and is unaffected by the constraint; the second is a correction of rank at most p that acts, through $J_g^\top$, on the downstream gradient expressed in semantic coordinates, $v := (A^\dagger)^\top \nabla \ell(h') \in \mathbb{R}^p$. For brevity we write $\nabla_{s'} \mathcal{L} := v$; the semantic-space gradient on which the constraint acts is then

$$\nabla_s \mathcal{L}(T(s)) := J_T(s)^\top \nabla_{s'} \mathcal{L} = \nabla_{s'} \mathcal{L} + J_g(s)^\top \nabla_{s'} \mathcal{L}.$$

Thus the backpropagated gradient in semantic space equals the residual-path gradient plus an additive correction whose matrix has nonnegative entries under Lemma 5.1. Nonnegativity rules out sign cancellation inside this correction; it does not by itself reduce its magnitude (indeed $J_T \geq I$ elementwise). All statements below concern this correction term.

A.4.1 SATURATION-INDUCED GRADIENT ATTENUATION

We now examine how saturation in monotone networks affects gradient magnitude. The idea is simple: a hidden unit whose activation is flat at its current pre-activation contributes nothing to the gradient that flows through it, so the more units sit in such a flat regime, the fewer directions an attacker can exploit. The assumption below fixes what “flat” means for the activations we use, and the lemma quantifies the contribution of nearly flat units.

Assumption (Assumption 5.2, restated). *The activation σ used inside g has bounded derivative and admits a saturated regime in the negative direction: $|\sigma'(z)| \rightarrow 0$ as $z \rightarrow -\infty$, and $|\sigma'|$ is non-decreasing on some tail $(-\infty, z^*]$. This holds exactly for ReLU, used by T5-small ($\sigma'(z) = 0$ for all $z < 0$, with $z^* = 0$), and for the softplus and logistic activations (with any $z^* \leq 0$); GELU, used by Pythia, satisfies it with $z^* = -\sqrt{2}$. ReLU, softplus, and GELU do not saturate in the positive direction, where $\sigma'(z) \rightarrow 1$; the logistic activation saturates on both tails, but only negative-tail saturation is used below.*

Lemma (Lemma 5.3, restated). *Let $T(s) = s + g(s)$ with $g(s) = W_2\sigma(W_1s + b_1) + b_2$, $W_1 \in \mathbb{R}^{m \times p}$ and $W_2 \in \mathbb{R}^{p \times m}$ elementwise nonnegative, and write $z_j(s) := (W_1s + b_1)_j$. Wherever σ is differentiable at $z(s)$,*

$$J_g(s) = \sum_{j=1}^m \sigma'(z_j(s)) W_2[:, j] W_1[j, :],$$

so that each hidden unit contributes a rank-one term to J_g weighted by its derivative. Let $S \subseteq \{1, \dots, m\}$ be a set of units with $\max_{j \in S} |\sigma'(z_j(s))| \leq \varepsilon$, and suppose $\|\nabla_{s'} \mathcal{L}\|_2 \leq G$. Then

$$\left\| \nabla_s \mathcal{L}(T(s)) - \left(I + \sum_{j \notin S} \sigma'(z_j(s)) W_2[:, j] W_1[j, :] \right)^\top \nabla_{s'} \mathcal{L} \right\|_2 \leq \varepsilon G \sum_{j \in S} \|W_2[:, j]\|_2 \|W_1[j, :]\|_2.$$

In particular, if σ satisfies Assumption 5.2, then along any sequence $\{s_k\}$ on which the pre-activations of the units in S diverge to $-\infty$ while $\nabla_{s'}\mathcal{L}$ remains bounded, the contribution of those units to $J_g(s_k)$, and hence to $\nabla_s\mathcal{L}(T(s_k))$, vanishes.

The proof is given in Appendix A.3.3.

This result shows that activation saturation can locally suppress the feed-forward branch’s own contribution $J_g^\top \nabla_{s'}\mathcal{L}$ to the backpropagated gradient, driving the sublayer Jacobian toward the identity of the residual path, without eliminating all gradient signal. Nonnegativity of the weights is not needed for the bound itself; its role is to prevent cancellation among the remaining active units (Lemma 5.1). Because of this, the suppression is a genuine attenuation whenever the downstream gradient is sign-coherent: if $\sigma' \geq 0$ and $\nabla_{s'}\mathcal{L} \geq 0$ (or ≤ 0) elementwise, every coordinate of $\nabla_{s'}\mathcal{L} + J_g^\top \nabla_{s'}\mathcal{L}$ is non-decreasing in absolute value in each $\sigma'(z_j)$, so saturating units can only shrink $\|\nabla_s\mathcal{L}(T(s))\|$ toward $\|\nabla_{s'}\mathcal{L}\|$. The condition $\sigma' \geq 0$ holds everywhere for ReLU, but for GELU it fails precisely on the saturated tail ($\text{GELU}'(z) < 0$ for $z < -0.752$), so this monotone-attenuation reading applies to the T5 track, while for the Pythia track only the norm bound of Lemma 5.3 is available. For mixed-sign downstream gradients, removing a unit’s contribution can increase the norm, and the lemma should be read as a statement about the feed-forward term alone.

A.4.2 PERSISTENCE OF SATURATION EFFECTS

The following observation captures a one-sided stability property of saturated units under monotone perturbations.

Lemma A.2 (Persistence of Saturated Units). *Let σ satisfy Assumption 5.2 with tail threshold z^* , and consider any first-layer pre-activation $z_j(s) = (W_1 s + b_1)_j$ within g , with $W_1 \geq 0$ elementwise. For $\varepsilon \geq 0$, call unit j ε -saturated at s if $z_j(s) \leq z^*$ and $|\sigma'(z_j(s))| \leq \varepsilon$ (for ReLU, with the convention $\sigma'(0) = 0$, 0-saturated means $z_j(s) \leq 0$). If unit j is ε -saturated at s' , then for any monotone semantic perturbation $\delta \leq 0$, i.e., for any $s' + \delta \leq s'$ in the product order on $\mathbb{R}^p$, the unit remains ε -saturated at $s' + \delta$.*

Proof. Let $z_j(s) := (W_1 s + b_1)_j$. Since $W_1 \geq 0$ elementwise and $\delta \leq 0$,

$$z_j(s' + \delta) = z_j(s') + (W_1 \delta)_j \leq z_j(s') \leq z^*.$$

Because $|\sigma'|$ is non-decreasing on $(-\infty, z^*]$ by Assumption 5.2, $|\sigma'(z_j(s' + \delta))| \leq |\sigma'(z_j(s'))| \leq \varepsilon$. Thus the unit remains ε -saturated at $s' + \delta$. For ReLU with $z^* = 0$ the argument is simply that $z_j(s') \leq 0$ implies $z_j(s' + \delta) \leq 0$. $\square$

Importantly, this lemma applies to individual hidden units rather than to the entire input gradient, and does not preclude other units from becoming active. Its direction is set by the activation: for the activations used here, saturation is preserved under monotone decreases of the semantic coordinates, whereas a monotone increase can re-activate a saturated unit.

A.4.3 IMPLICATIONS FOR GRADIENT-BASED ATTACKS

Gradient-based adversarial attacks such as HotFlip rely on persistent, high-magnitude gradients with respect to token embeddings to identify impactful discrete perturbations. The preceding analysis is conditional: it does not show that the constraint *produces* saturation, and whether training under nonnegativity places pre-activations of g in saturated regimes is an empirical question that we do not test here. What it shows is that, if hidden units of g are saturated, their contribution to the semantic gradient is small (Lemma 5.3), that this state is stable under monotone decreases of the semantic coordinates (Lemma A.2), and that the contributions of the remaining active units cannot cancel one another (Lemma 5.1; this last property requires $\sigma' \geq 0$ and therefore holds exactly for the ReLU FFNs of the T5 track and only up to the small negative-derivative region of GELU for the Pythia track). Together these properties reduce the number and the variability of the directions through which a gradient-based attacker can move the sublayer output.

Scope. The analysis concerns the feed-forward branch of a single sublayer. The gradient with respect to token embeddings that HotFlip uses is a product of Jacobians across all sublayers, in

which the residual identity, attention, and normalization paths are unconstrained; even if every feed-forward correction term vanished, this product would reduce to that of the unconstrained paths. The mechanism is therefore local, and the controls of Section 6.4 (random and gradient-free attacks) are needed to establish that the observed robustness is not an artifact of gradient-based optimization.

While monotone feed-forward networks are not dynamical systems in the formal sense, their behavior is reminiscent of classical results in monotone dynamical systems, where cooperative interactions restrict the effective directions along which trajectories evolve. This analogy is intended as intuition rather than a formal guarantee: monotonicity does not prevent all adversarial attacks, nor do gradients vanish globally. Instead, our analysis identifies a local attenuation effect that weakens gradient-based optimization strategies, offering a plausible mechanism for the empirical robustness improvements observed in Section 6.

A.5 TRAINING AND EVALUATION PROTOCOL

Both baseline and monotone models are fine-tuned from the same pretrained T5-small checkpoint under identical optimization settings. We use the AdamW optimizer with learning rate 5×10^{-5} and weight decay 0.01, a batch size of 4, and gradient clipping with norm 1.0. To accommodate the constrained parameterization, the monotone model employs an extended warmup phase of 15% of training steps, compared to 10% for the baseline; we did not ablate this choice. Both models are trained for 7 epochs with linear learning-rate decay after warmup and receive the same total training budget; the checkpoint with the lowest validation loss is used for all downstream evaluation.

Training is conducted on the DialogSum, HighlightSum, and arXiv abstract datasets, comprising 242,774 examples in total, with 8,296 validation examples drawn from the corresponding validation splits. Evaluation is performed on two held-out benchmarks using their full test sets: CNN/DailyMail (11,490 test examples) and XSUM (11,333 test examples; one example with an empty field is dropped by the loader). Neither CNN/DailyMail nor XSUM is used in training, so both evaluations measure out-of-distribution generalization.

At inference time, decoding is performed using beam search with 4 beams and a length penalty of 1.2, enforcing a minimum generation length of 10 tokens and a maximum length of 80 tokens. We additionally apply no-repeat n -gram blocking with $n = 3$ to discourage degenerate repetitions. All decoding parameters are fixed across models and evaluation sets. ROUGE-1, ROUGE-2, and summary-level ROUGE-L (rougeLsum) F_1 scores are computed using the rouge-score library with Porter stemming. Within each seed we report bootstrap-based 95% confidence intervals using 1,000 resamples and assess statistical significance via per-example paired t -tests between the baseline and monotone models, with Bonferroni correction across the three ROUGE variants. Effect sizes are reported using Cohen’s d .

A.6 SEEDS, STATISTICAL TESTS, AND COMPUTE

To evaluate robustness to training stochasticity, the T5 training and evaluation pipeline is repeated using five random seeds (42, 1337, 2024, 8888, and 12345), controlling randomness across Python, NumPy, and PyTorch. Summarization quality, training dynamics, and the order-preservation diagnostic are reported as the mean and standard deviation across all five seeds; universal-trigger attacks are reported across four seeds (1337, 2024, 8888, 12345; the seed-42 checkpoints were not evaluated under the final trigger protocol); the HotFlip evaluation in Table 2 uses the seed-2024 models, corroborated by an earlier evaluation of the seed-42 models. Across-seed differences are assessed using paired t -tests on seed-level means (baseline vs. monotone ROUGE-L: $t = 14.9$ on CNN/DailyMail and $t = 13.4$ on XSUM with 4 degrees of freedom), complemented by per-example paired t -tests within each seed. This dual evaluation captures both test-time variability and variance induced by random initialization and data ordering. All experiments are conducted using deterministic algorithms where available, on NVIDIA A100 (40GB and 80GB), H200 (141GB), and RTX PRO 6000 (96GB) GPUs under the SLURM workload manager, using PyTorch builds with CUDA 11.8 (A100) or CUDA 12.8 (H200, RTX PRO 6000) and Transformers 4.30 or later.

A.7 FOUNDATION MODEL TRAINING PROTOCOL

The decoder-only track fine-tunes Pythia-1.4B (Biderman et al., 2023) on the uncopyrighted subset of the Pile (Gao et al., 2020), with held-out evaluation texts drawn from a disjoint streaming sample. Constraint-scope screening (Table 7) was run on seed 42, the MLP-in scope under the full budget described below and the MLP-both and MLP-in + Attn-out scopes under a reduced budget of 30,000 training samples at sequence length 1024 for 3 (baseline) and 4 (monotone) epochs; full-budget validation of the selected MLP-both scope (Table 3) trains the baseline for 5 epochs and the monotone model for 10 epochs on each of three seeds (42, 1337, 2024). The extended monotone budget parallels the extended warmup used in the T5 track and compensates for the constrained parameterization’s slower recovery. Both recovery runs use AdamW with weight decay 0.01, linear warmup followed by linear decay, gradient clipping at norm 1.0, an effective batch of 32 sequences (8 per step with 4 accumulation steps) at sequence length 2048, and bfloat16 weights with gradient checkpointing; the learning rate and warmup fraction are 10^{-5} and 10% for the baseline and 5×10^{-5} and 20% for the monotone model. Full-budget runs train on the first 100,000 documents of the streamed training split (90%/10% train/validation) and retain the checkpoint with the lowest validation perplexity. HotFlip evaluation uses 200 held-out examples with up to 10 token flips per example; UAT evaluation uses coordinate ascent with 3 restarts and 50 iterations, optimizing triggers on 80 texts and evaluating on 120 disjoint texts. Universal-trigger optimization and evaluation (including the trigger-transfer replay and the order-preservation probe) run with the models in bfloat16 precision; HotFlip, the substitution-transfer and gradient-free controls, and held-out perplexity are computed in float32. All foundation-model experiments are executed on NVIDIA A100 GPUs with 80GB memory under the SLURM workload manager, with per-stage checkpointing to permit resumption across scheduler preemptions.

A.8 ATTACK, CONTROL, AND DIAGNOSTIC PROTOCOLS

This appendix reproduces the full protocol descriptions summarized in Section 4.

Universal Adversarial Triggers. Universal adversarial triggers (UATs) (Wallace et al., 2019) consist of short, input-agnostic token sequences that, when prepended to an input, maximize the model’s loss. We optimize five-token triggers using coordinate-wise search with five random restarts and 100 iterations. Candidate tokens are drawn from a restricted pool of about 4,000 SentencePiece ids (ids 1000–4999) augmented with a short hand-picked list of negation and discourse words (e.g., *not*, *never*, *ignore*), punctuation strings, and numerals; each iteration re-samples three of the five trigger positions, scores 50 random candidates per position on a random 30-example subset, and the trigger with the highest mean loss on the full optimization set is kept. Triggers are optimized on 1,000 held-out CNN/DailyMail validation examples and evaluated on 1,500 disjoint test examples; the cross-model transfer matrix is computed on the first 100 evaluation examples. The trigger is inserted at the start of the encoder input, ahead of the `summarize:` prefix, and the concatenation is then truncated *from the left* to the 512-token encoder budget. For articles longer than about 505 tokens the trigger is therefore partially or entirely discarded and the attacked input coincides with the clean input; because many CNN/DailyMail articles exceed this length, the universal-trigger results on T5 reflect only the shorter articles and should be read as a lower bound on trigger effectiveness.

HotFlip Attacks. HotFlip attacks (Ebrahimi et al., 2018) identify vulnerable input positions by computing gradients of the loss with respect to token embeddings and replacing selected tokens to maximize loss increase. We attack the five positions (special tokens excluded) with the largest gradient norms, selecting the replacement at each position by the dot product between the gradient and the candidate embedding.

Robustness Metrics. Robustness is quantified using the relative increase in the model’s cross-entropy loss under attack (*degradation*, averaged over examples), the attack success rate (the fraction of examples whose degradation exceeds 10%), and statistical significance assessed via independent-samples *t*-tests on the per-example degradations. Because relative degradation is normalized by each model’s own clean loss, we also report the absolute loss increase in nats, which is insensitive to differences in clean loss between models. For UATs we report the relative increase in negative log-likelihood (NLL) and ROUGE changes under the trigger, and analyze cross-model transferability of adversarial triggers.

Order-preservation diagnostic. To test whether sublayer monotonicity translates into order preservation along the semantic directions that motivate it, we construct a templated corpus of ordered prompt pairs: 13 semantic axes (severity, safety constraints, certainty, urgency, and others, as in Section 1) instantiated on 18 topics as four-level chains, yielding 702 adjacent-level pairs (weaker $\leq$ stronger), split at the chain level into 507 probe-fitting and 195 disjoint evaluation pairs. For each model and pooling we fit $p = 64$ probe directions A on final-layer representations of the fitting split, each row being an ℓ_2 -regularized logistic-regression direction for stronger-versus-weaker fit on a bootstrap resample of the fitting pairs, so that the 64 rows are correlated readouts of one underlying ordering signal rather than 64 distinct axes. For every evaluation pair and every layer we then report the fraction of the 64 probe coordinates for which $Ah_{\text{weak}} \leq Ah_{\text{strong}}$, with bootstrap confidence intervals, under both mean and last-token pooling. This instrument is fit post hoc and plays no role in training.

Decoder-only track. The T5 experiments above concern an encoder-decoder architecture. To determine whether the monotonicity–robustness relationship generalizes to the decoder-only architectures that dominate contemporary foundation models, we conduct a second experimental track using Pythia-1.4B (Biderman et al., 2023), a 24-layer autoregressive language model from the GPT-NeoX family. Models are fine-tuned and evaluated on the Pile (Gao et al., 2020) (uncopyrighted subset), and clean performance is measured by *perplexity*—the exponentiated mean negative log-likelihood per token—on a held-out sample, the standard intrinsic quality metric for autoregressive language models.

Because decoder-only blocks lack the encoder-decoder bottleneck of T5, it is not obvious which sublayers must be constrained for monotonicity to bind. Each GPT-NeoX block contains an attention sublayer with an output projection and a feed-forward (MLP) sublayer with an input projection ($W_{\text{in}} \in \mathbb{R}^{4d \times d}$) and an output projection ($W_{\text{out}} \in \mathbb{R}^{d \times 4d}$). We therefore compare three *constraint scopes*, each applying the softplus reparameterization of Section 3 directly to the designated weight matrices in all 24 layers (enforcing elementwise nonnegativity, i.e., the coordinatewise order, with $A = I$ exactly as in the T5 track):

- **MLP-in:** constrain only W_{in} ;
- **MLP-both:** constrain both W_{in} and W_{out} , the direct analogue of the T5 scheme;
- **MLP-in + Attn-out:** constrain W_{in} and the attention output projection.

Two caveats apply. First, the GPT-NeoX MLP uses the GELU activation, which is not globally non-decreasing ($\text{GELU}'(z) = \Phi(z) + z\phi(z) < 0$ for $z < -0.752$, with $\min_z \text{GELU}'(z) \approx -0.13$ attained at $z = -\sqrt{2}$), so Proposition 3.4 does not apply exactly: the constrained Pythia FFNs are nonnegative-weight networks that are monotone up to this small non-monotone region of the activation, whereas the T5 track (ReLU) satisfies the proposition exactly. We retain GELU for compatibility with the pretrained weights and treat the Pythia track as an empirical test of the same weight constraint rather than as an exactly monotone model. Second, only the MLP-both scope constrains every weight matrix of the FFN branch; MLP-in leaves W_{out} free and is included as an ablation rather than as a monotone model, and the Attn-out scope constrains a matrix outside the FFN. After imposing constraints, each model undergoes *recovery training*—fine-tuning on the Pile to re-establish effective representations under the constrained parameterization—with the unconstrained baseline fine-tuned on the same data. The two recovery runs differ in learning rate (10^{-5} for the baseline, 5×10^{-5} for the monotone model), warmup fraction (10% versus 20%), and length (5 versus 10 epochs), the monotone model receiving the larger budget to compensate for its disrupted initialization (Appendix A.7). Constraint scopes are first screened on a single seed (MLP-in under the full recovery budget, the two broader scopes under a reduced budget, which is why their baseline entries in Table 7 differ; the scope comparison is therefore indicative rather than budget-matched); the selected scope is then validated under the full recovery budget across three seeds (42, 1337, 2024). **Attacks.** Robustness is assessed with causal-LM analogues of the T5 attacks (separate implementations that follow the same attack logic but differ in candidate handling and in the loss that is attacked): HotFlip with $n = 200$ examples and up to 10 token flips chosen by an unrestricted argmax over the vocabulary (success threshold 15%), and ten-token UATs optimized by coordinate ascent with three restarts and 50 iterations on 80 texts and evaluated on 120 disjoint texts, with 20 sampled candidates per position per sweep drawn from a fixed pool of 100 token ids (the lowest-numbered ids of a candidate set comprising vocabulary ids 1000–4999 and a short list of negation

Table 5: Training dynamics (mean $\pm$ s.d. over five training seeds; both models are trained for 7 epochs).

Model	Train loss (ep. 1)	Train loss (ep. 7)	Val. loss (ep. 1)	Val. loss (ep. 7)
Baseline	2.94 ± 0.00	2.40 ± 0.00	2.43 ± 0.00	2.21 ± 0.00
Monotonic	4.99 ± 0.02	2.73 ± 0.00	2.92 ± 0.01	2.51 ± 0.00

Table 6: Summarization quality on the full XSUM test set ($n = 11,333$). Values are ROUGE $F_1 \times 100$, mean $\pm$ s.d. over five training seeds.

Model	ROUGE-1	ROUGE-2	ROUGE-L
Standard	20.1	2.9	13.5
Baseline	20.1 ± 0.0	3.1 ± 0.0	13.1 ± 0.0
Monotonic	19.3 ± 0.1	2.9 ± 0.0	12.8 ± 0.0

and punctuation tokens). **Loss convention.** Attack effect is measured as the relative increase in the mean per-token negative log-likelihood (NLL) of the perturbed sequence. In this track the loss is the causal-LM loss of the perturbed input itself, so the trigger tokens (and, for the substitution attacks below, the substituted tokens) are among the prediction targets; this differs from the T5 track, where the loss is that of the fixed reference summary given the perturbed source, and it means that part of every reported NLL increase reflects the low likelihood of the inserted tokens themselves rather than their effect on the remaining text. Because both models are scored identically, comparisons between them remain valid, but absolute degradation figures in this track are inflated by this effect. **Controls.** To distinguish genuine robustness from gradient masking, we add three controls on the full-budget models. *Substitution transfer* replays the token substitutions found by HotFlip on one model verbatim against the other, yielding the full crafted-on $\times$ evaluated-on matrix (and analogously for UAT triggers, evaluated on 900 held-out texts). A *random-substitution control* applies the same number of substitutions at uniformly random positions with replacement tokens drawn uniformly from the same restricted candidate vocabulary used by the trigger search, involving no model or gradient. A *gradient-free query attack* chooses positions at random and, at each of the 10 positions, keeps whichever of 20 candidate tokens from that vocabulary most increases the target model’s own loss (greedy coordinate search on 50 texts, $10 \times 20 + 2$ forward passes per example).

A.9 ADDITIONAL RESULTS

Training dynamics and XSUM. Table 5 reports per-epoch training and validation losses for both models (mean $\pm$ s.d. over the five seeds); the monotone model recovers rapidly, reducing training loss by 1.94 points in the first two epochs (from 4.99 to 3.05), followed by slower asymptotic convergence with diminishing returns in later epochs. Table 6 reports ROUGE on the full XSUM test set; both are discussed in Section 6.1.

HotFlip on the seed-42 checkpoints. An earlier evaluation of the seed-42 checkpoints on 100 CNN/DailyMail test examples gave average degradations of 5.2, 16.2, and 21.3 percent and success rates of 19, 63, and 69 percent for the monotonic, baseline, and standard models respectively, matching the seed-2024 results of Table 2.

Generation length. Analysis of generation behavior reveals that both fine-tuned models produce longer outputs on CNN/DailyMail than the pretrained T5 model (mean lengths of 75.4 tokens for the baseline and 74.9 for the monotonic model vs. 62.3 for the standard model, against a mean reference length of 78.3 tokens; five-seed means), reflecting adaptation to the distributional properties of the summarization training data. The monotonic model exhibits somewhat higher length variance (s.d. 9.5 tokens) than the baseline (6.6 tokens), though both are below that of the standard model (12.0 tokens). Importantly, the brevity penalty of the monotonic model (0.955) is close to that of the baseline (0.962), indicating that output length is well-calibrated to reference summary length and that the constraints do not introduce systematic length bias.

Constraint-scope screening on Pythia-1.4B. Table 7 reports the screening discussed in Section 6.4; MLP-in was screened under the full training budget and the two broader scopes under

Table 7: Constraint-scope screening on Pythia-1.4B (seed = 42). PPL denotes held-out Pile perplexity; SR denotes HotFlip attack success rate ($n = 200$, 10 flips, 15% threshold), and SR drop its reduction in percentage points (pp). MLP-in was screened under the full training budget, the other scopes under a reduced budget.

Constraint scope	PPL base	PPL mono	SR base $\rightarrow$ mono	SR drop
MLP-in	6.67	24.32	71.5% $\rightarrow$ 83.5%	−12.0 pp
MLP-both	7.06	47.99	75.0% $\rightarrow$ 44.5%	+30.5 pp
MLP-in + Attn-out	7.06	88.17	75.0% $\rightarrow$ 35.5%	+39.5 pp

the reduced budget of Appendix A.7, which is why their baseline entries differ. The diminishing return of additionally constraining the attention output projection is consistent with attention serving as a secondary leak path.

Transfer and gradient-free controls (full discussion). A natural concern is that a constrained model merely masks gradients, so that gradient-based attacks fail while the model remains vulnerable to perturbations found by other means (Athalye et al., 2018; Carlini et al., 2019; Tramèr et al., 2020). Table 4 reports the controls of Section 4 on the three full-budget seeds; its HotFlip rows re-run the attack of Table 3 inside the transfer pipeline, where the baseline success rates (68.5–69.0%) differ from Table 3 by at most 0.5 points, and the random control and query attack are model-agnostic and target-scored, respectively. Three observations follow. First, the monotone model’s advantage persists under every gradient-free attack: random substitutions raise the baseline’s loss by 64.6 ± 0.3 percent (success rate 73.0 percent) but the monotone model’s by 17.9 ± 1.5 percent (36.5 percent), and the query attack—the strongest attack we ran—degrades the baseline by 112 ± 4 percent versus 34 ± 3 percent for the monotone model (success rates 77.3 vs. 50.0 percent). Since neither control uses gradients, this gap cannot be explained by gradient masking.

Second, attacks do not transfer symmetrically: substitutions crafted on the baseline degrade the monotone model by only 15.4 percent, less than random substitutions do, whereas substitutions crafted on the monotone model degrade the baseline by 65.0 percent, as much as random ones. Universal triggers behave the same way (Appendix A.11): on the 900-text transfer set, a trigger learned on the baseline raises its NLL by 17.8 percent but the monotone model’s by only 6.3 percent, while a trigger learned on the monotone model raises the baseline’s NLL by 13.3 percent (the on-model figures differ slightly from Table 3, which uses the 120-text evaluation set).

Third, in absolute terms the gaps are narrower than the relative figures suggest (random: 0.66 vs. 0.91 nats; query: 1.17 vs. 1.39 nats; HotFlip: 0.77 vs. 0.82 nats), because the monotone model starts from a clean NLL roughly 1.9 times higher. The monotone Pythia model is therefore less sensitive to token substitutions of every kind we tested, but a substantial part of its relative advantage reflects its higher clean loss; the encoder–decoder results, where the absolute gap is 2.5-fold, are not subject to this caveat to the same degree. Finally, we note that on the baseline, HotFlip’s gradient-guided substitutions (55.2 percent) are less damaging than random ones (64.6 percent), indicating that the first-order approximation underlying HotFlip is a weak guide for this model and that the query attack is the more informative reference attack in this setting.

A.10 ATTRIBUTION ABLATION: CONSTRAINT VERSUS INITIALIZATION

The monotone parameterization changes two things relative to the pretrained model: it imposes the nonnegativity constraint, and it initializes the FFN weights at $|W_{\text{pre}}|$, discarding their sign pattern. To attribute the clean-quality cost between these two factors we train two additional T5-small models at seed 42 under the protocol of Appendix A.5. The *sign-frozen* arm uses $W = \text{sign}(W_{\text{pre}}) \odot \phi(V)$ with the sign pattern held fixed, so the reparameterization and magnitude initialization are identical to the monotone model but the sublayer is not constrained to be monotone; the *abs-init-free* arm uses $W = V$ with V initialized at $|W_{\text{pre}}| + \epsilon$ and trained without any constraint. Table 8 reports their training dynamics. The sign-frozen arm converges to within 0.06 of the baseline validation loss, whereas the abs-init-free arm converges to the same validation loss as the monotone model. The clean-quality cost of the monotone model is therefore attributable to the loss of the pretrained sign structure at initialization rather than to the reparameterization or to the constraint itself. Whether the robustness gain is likewise attributable to the constraint rather than to the initialization requires the corresponding robustness evaluation of these two arms, which we leave to future work.

Table 8: Attribution ablation on T5-small (seed 42; 7 epochs). Validation loss at epoch 1 and epoch 7.

Arm	Parameterization	Constraint	Val. loss (ep. 1)	Val. loss (ep. 7)
Baseline	W (pretrained)	none	2.43	2.21
Sign-frozen	$\text{sign}(W_{\text{pre}}) \odot \phi(V)$	none	2.51	2.27
Abs-init-free	$V, V_0 = W_{\text{pre}} + \epsilon$	none	2.91	2.51
Monotone	$\phi(V), V_0 = \phi^{-1}(W_{\text{pre}} + \epsilon)$	$W \geq 0$	2.92	2.51

A.11 UNIVERSAL TRIGGER TRANSFER

Table 9 reports the cross-model transfer of learned universal triggers on Pythia-1.4B, evaluated on 900 held-out texts; the on-model (diagonal) entries therefore differ slightly from Table 3, which uses the 120-text evaluation set. The baseline is vulnerable to triggers learned on either model, whereas the monotone model is substantially less affected by the baseline’s trigger than by its own. On T5, where universal triggers are weak against every model, on-model ROUGE-L changes under the triggers are within ± 0.3 points for the fine-tuned models in every seed, and the off-diagonal entries of the corresponding ROUGE-L transfer matrix (computed on the first 100 evaluation examples) average within ± 0.35 points and never exceed 0.75 points in any of the four seeds.

Table 9: Universal-trigger transfer on Pythia-1.4B: relative NLL increase (mean $\pm$ s.d. over seeds 42, 1337, 2024) when the trigger learned on the row model is prepended to inputs of the column model.

Trigger learned on	Evaluated on baseline	Evaluated on monotone
Baseline	$+17.8 \pm 0.7\%$	$+6.3 \pm 0.3\%$
Monotone	$+13.3 \pm 0.5\%$	$+11.1 \pm 3.7\%$

A.12 EXTENDED DISCUSSION

This work provides empirical evidence, together with a conditional theoretical mechanism, that monotonicity is a promising architectural bias for improving the robustness of modern language models. By selectively enforcing monotonicity in feed-forward sublayers, we consistently reduce vulnerability to token-level adversarial attacks while largely preserving task performance in encoder-decoder models. These improvements require neither additional data nor modified training objectives. Instead, they arise from a favorable architectural trade-off in which modest reductions in task performance yield substantial improvements in adversarial robustness.

Our theoretical analysis offers insight into why these improvements arise despite the absence of formal end-to-end robustness guarantees. Viewing a language model as a high-dimensional system whose semantic representations evolve through successive nonlinear transformations, end-to-end monotonicity would constrain how perturbations propagate across layers by enforcing directional consistency: strengthening information or constraints could not induce regressions in downstream representations, and worst-case effects would be easier to characterize because they would occur at semantic extremes rather than at intermediate states. Our models, however, are monotone only within the feed-forward branch of each sublayer, and Section 6.3 shows that the trained networks are not more order-preserving end to end, so this reasoning is a design heuristic rather than a property we can claim for the trained models.

The empirical results further suggest that feed-forward sublayers play a central role in robustness. While attention performs contextual reasoning and semantic aggregation, feed-forward networks are primarily responsible for nonlinear transformation and feature amplification. Enforcing monotonicity at this stage constrains how semantic information is refined after context has been established, without limiting attention or global information flow. This selective constraint also changes the local gradient structure of the constrained sublayers (Section 5), which we conjecture reduces the gradients exploited by attacks such as HotFlip.

These observations are consistent with classical monotone systems theory, where structural monotonicity limits the propagation of perturbations and enables more tractable analysis of high-

dimensional systems. Although Transformers are not dynamical systems in the formal sense, this perspective provides useful intuition for understanding the behavior of monotonic components within the network.

Two further results sharpen this picture. First, the gradient-free controls on Pythia-1.4B show that the robustness advantage is not confined to gradient-based attacks: the monotone model is also less sensitive to random substitutions and to a query-based attack that never computes a gradient. The gradient-attenuation mechanism of Section 5 therefore describes one symptom of a more general reduction in sensitivity to token-level perturbations, rather than a masking of gradients that a stronger attacker could circumvent. At the same time, the absolute-loss view shows that part of the monotone Pythia model’s relative advantage reflects its higher clean loss, so relative degradation should be reported together with absolute degradation whenever constrained and unconstrained models differ substantially in clean performance.

Second, the order-preservation diagnostic shows that per-sublayer, coordinatewise monotonicity does not by itself make the trained network more order-preserving along learned semantic directions; if anything, the monotone T5 encoder is less order-preserving at intermediate depth. The semantic-ordering motivation of Section 1 should thus be understood as the design principle behind the constraint, whereas the mechanism that the experiments support is the local sensitivity reduction of the constrained sublayers. Closing this gap—for instance by imposing the constraint in a learned semantic basis A as in Definition 3.5, rather than in the coordinatewise basis—is a natural next step.

The decoder-only experiments further reveal that robustness depends on architectural signal paths rather than individual layers. The residual stream provides unconstrained routes around any single constrained matrix: constraining only the feed-forward input projection is ineffective on Pythia-1.4B, while constraining both feed-forward projections, as in the T5 track, restores much of the robustness benefit. In a decoder-only model, autoregressive self-attention connects every position to all preceding positions without passing through any constrained layer, providing a bypass for adversarial signal; we conjecture that the constrained feed-forward sublayers must therefore absorb a larger share of each block’s transformation to close this bypass, which would explain the steeper expressivity penalty (the held-out perplexity factor is 5.61 on average, range 5.21–6.25 across seeds). Monotonicity should therefore be viewed as a property of the paths through which adversarial signals propagate, providing practical guidance for where constraints should be applied in different Transformer architectures.

The quality–robustness trade-off also depends on architecture. In encoder-decoder models, the reduction in expressivity introduced by monotonicity has only a modest effect on task performance, while yielding substantial gains in robustness. In decoder-only architectures, the same constraint scope incurs a considerably larger perplexity cost that full-budget recovery training does not eliminate, resulting in a significantly steeper quality–robustness trade-off. Reducing this cost through improved initialization, specialized optimization strategies, or selective constraint placement remains an important direction for future work. More broadly, monotonicity differs from conventional regularization by constraining *how* representations evolve rather than simply penalizing their magnitude, encouraging predictable behavior instead of uniformly smoother representations.

Our study is limited to models of modest scale (T5-small and Pythia-1.4B) and does not provide formal robustness certificates. Extending the theoretical analysis toward stronger guarantees, scaling the approach to larger models, and narrowing the quality gap in decoder-only architectures remain important directions for future research. Nevertheless, the combination of theoretical insight and consistent empirical improvements across two Transformer families suggests that monotonicity is a practical and principled architectural prior for improving the robustness of large language models.